

Chapter

07

7장에서는 클로드 코드의 AI 서브에이전트 기능을 활용해 혼자서도 실제 개발팀처럼 일하는 방법을 배웁니다. 먼저 코드리뷰, 최적화, UX 디자인 등 역할을 수행하는 서브에이전트의 개념과 생성 방법을 익히고, 여러 에이전트가 협업해 프로젝트를 개선하는 과정을 실습합니다.

이후 제품 기획 관리자, 백엔드 및 프론트엔드 개발자, 품질 보증 엔지니어, AI 통합 전문가로 구성된 AI 개발팀을 구축해 실제 애플리케이션을 자동으로 완성합니다.

각 에이전트가 역할을 분담하면 더 체계적으로 프로젝트를 진행할 수 있고, 혼자서는 놓치기 쉬운 부분도 보완할 수 있습니다.

클로드 코드 AI 에이전트로 개발팀 구성하기

학습목표

- 클로드 코드의 에이전트 기능을 이해하고 생성할 수 있습니다.
- 각 에이전트에게 전문 역할과 권한을 부여할 수 있습니다.
- 여러 에이전트를 협업시켜 복잡한 프로젝트를 완성할 수 있습니다.
- 실제 개발팀처럼 AI 에이전트 팀을 구성하고 운영할 수 있습니다.

07-1

클로드 코드의 AI 에이전트 이해하기

핵심 키워드

에이전트

AI 서브에이전트

/agents

코드 리뷰

최적화

UX 디자인

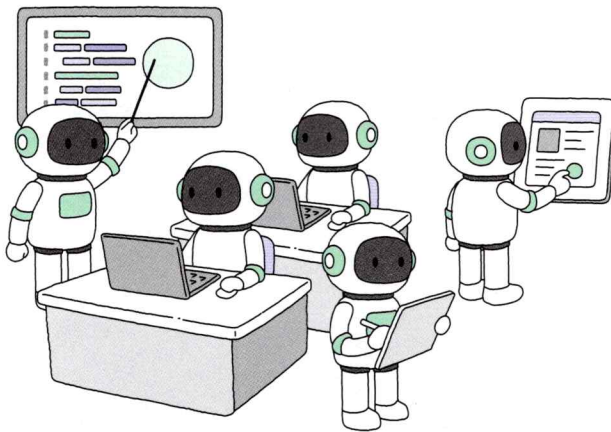
이번 절에서는 클로드 코드의 AI 에이전트 개념과 역할 분담 방식을 배웁니다. 코드 리뷰, 최적화, UX 디자인 등 각 분야를 맡는 서브에이전트 생성 및 관리 절차를 실습하며 권한과 모델 설정 방법을 익힙니다. 마지막으로 여러 에이전트가 직렬 또는 병렬로 협업해 프로젝트를 개선하는 과정을 통해 AI 기반 팀 워크플로를 체험합니다.

시작하기 전에

에이전트 agent 는 영어로 ‘대리인’, 즉 ‘대신 일하는 사람’이라는 뜻입니다. 사람이 매번 세부 지시를 내리지 않아도 정해 둔 목표와 규칙, 사용할 도구, 그리고 필요한 맥락을 바탕으로 스스로 판단하고 작업을 수행합니다. 즉 개발자가 ‘무엇을 달성할지’를 지정하면, 에이전트가 ‘어떻게 할지’를 결정합니다.

복잡한 프로젝트를 혼자서 진행하다 보면, 코드 작성부터 디자인, 테스트까지 해야 하는 일이 반복되어 버겁기도 하죠. 클로드 코드의 AI 서브에이전트를 활용하면 이런 과정을 효율적으로 분담할 수 있습니다. 각 에이전트가 맡은 역할을 수행하면서 반복 작업을 줄이고, 놓치기 쉬운 부분을 보완합니다.

프로젝트 개발실

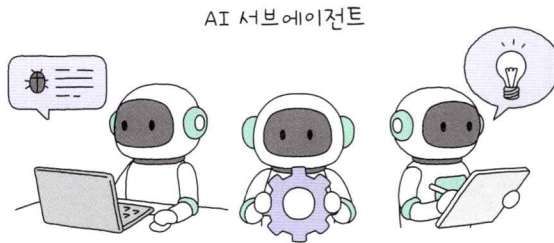


나의 첫 에이전트, 코드 리뷰어

클로드 코드에서는 작업을 맡길 수 있도록 전문화된 **AI 서브에이전트** subagent를 생성하고 사용할 수 있습니다. AI 서브에이전트는 특정 유형의 작업을 전문적으로 수행하는 개발 도구므로, 사용자가 직접 시스템 프롬프트를 정의하는 등 커스터마이징할 수 있어 효율적인 문제 해결을 돕습니다.

클로드 코드 서브에이전트

복잡한 프로젝트를 혼자 진행하다 보면 여러 역할을 동시에 수행해야 해서 힘들 때가 있습니다. 코드도 짜고, 디자인도 하고, 버그도 찾아야 하죠. 이때 클로드 코드 안에 여러 AI 서브에이전트를 두고 각 에이전트가 맡은 역할에 맞게 전문적인 일을 담당하게 할 수 있습니다. 마치 실제로 회사를 운영하는 것처럼 코드 리뷰어 에이전트는 코드에 버그가 있는지 검토하고, 최적화 에이전트는 성능을 개선하며, UX 디자인 에이전트는 사용자 경험을 향상시킵니다.



이번에는 클로드 코드에서 코드 리뷰 전용 AI 서브에이전트를 만들어 보며, 에이전트의 개념과 생성 절차를 배워 볼 것입니다.

클로드 코드가 직접 제공하는 서브에이전트는 Sonnet, Opus 등 각각 다른 AI 모델을 선택하거나 에이전트별로 서로 다른 파일 접근 권한을 가질 수 있습니다. 따라서 작업 맥락 context을 유지하면서 여러 에이전트가 순차적으로 협업할 수 있죠. 특히 읽기 전용, 쓰기 권한 등을 세밀하게 설정할 수 있어 안전하고 체계적인 작업 분담이 가능합니다.

+ 여기서 잠깐 서브에이전트 vs 커스텀 명령어

5장에서 배운 **커스텀 명령어**와 AI 서브에이전트는 얼핏 비슷해 보이지만, 그 역할에는 차이가 있습니다. 커스텀 명령어가 반복 작업을 빠르게 실행하는 단축키라면, 에이전트는 독립적인 AI 전문가에 가깝지요.

에이전트는 각자 다른 AI 모델을 선택할 수 있고, 파일 접근 권한을 개별 설정할 수 있으며, 다른 에이전트와 협업할 수도 있습니다. 예를 들어 코드 리뷰어 에이전트는 읽기 전용으로, 수정 에이전트는 쓰기 권한을 갖도록 설정해 역할을 명확히 분리할 수 있습니다.

코드 리뷰어 에이전트 생성하기

앞서 ‘냉장고를 부탁해’ 애플리케이션을 만든 ‘Study-04’ 폴더에 마우스를 대고 오른쪽 버튼을 누른 뒤, [터미널에서 실행] 메뉴를 클릭합니다. 명령 프롬프트가 실행되면 ‘claude’를 입력해 클로드 코드를 시작합니다.

01 터미널에 ‘/agents’ 명령어를 입력해 서브에이전트 인터페이스를 여세요.

```
> /agents
/agents
  Manage agent configurations
```

02 그러면 이와 같이 새로운 에이전트를 만들 수 있는 화면이 나타납니다. ‘**2** Create new agent’를 선택한 상태에서 [Enter] 키를 누릅니다.

화면 해설

1 에이전트를 찾을 수 없음

2 > 새 에이전트 만들기

등록된 에이전트가 없습니다. 클로드가 작업을 위임할 수 있는 전문화된 서브에이전트를 생성하세요. 각 서브에이전트는 자체 컨텍스트 창, 커스텀 시스템 프롬프트, 전용 도구 세트로 구성됩니다.

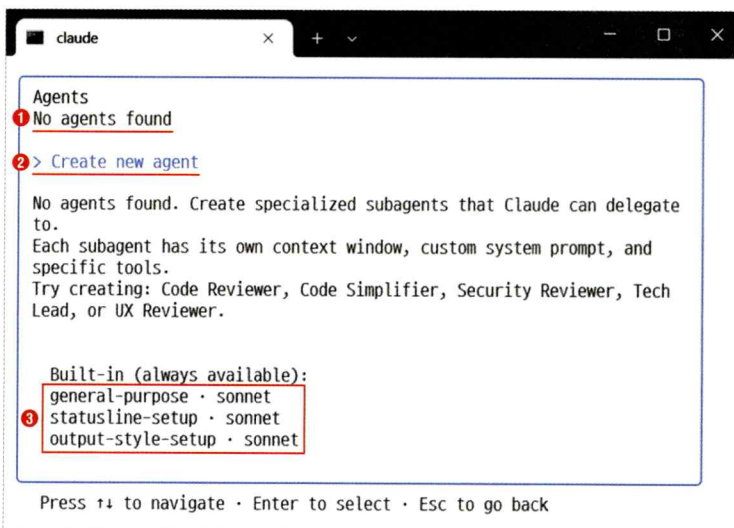
다음과 같이 만들어 보세요: 코드 리뷰어, 코드 단순화, 보안 점검, 기술 리드, UX 리뷰어

3 항상 사용 가능한 내장 에이전트:

일반 사용 목적 에이전트 · sonnet

상태 표시줄 전용 에이전트 · sonnet

출력 형식 관리 에이전트 · sonnet



note 내장 에이전트는 프로젝트를 진행하면서 필요한 작업이 있을 때 자동으로 수행하거나, 필요할 때만 활성화되는 지능형 도우미입니다. 상황에 따라 스스로 동작하거나 대기하며, 프로젝트를 효율적으로 진행할 수 있도록 지원합니다.

03 현재 프로젝트에서만 적용될 서브에이전트를 생성하기 위해서 '1. Project (.claude/agents/)'를 선택하겠습니다.

1. Project (.claude/agents/): 서브에이전트가 지금 실행 중인 프로젝트에서만 활동합니다.
2. Personal (~/.claude/agents/): 서브에이전트가 사용자의 전체 프로젝트에서 활동합니다.

```
> /agents

Create new agent
Choose location

> 1. Project (.claude/agents/)
   2. Personal (~/.claude/agents/)

Press ↑ to navigate · Enter to select · Esc to cancel
```

사용자 수준(Personal) 에이전트는 모든 프로젝트에서 사용할 수 있어 편리하지만, 프로젝트별로 다른 설정이 필요한 경우 충돌할 수 있습니다. 처음에는 프로젝트 수준(Project) 에이전트로 시작하고, 자주 사용하는 에이전트만 사용자 수준으로 만드는 것을 추천합니다.



04 에이전트의 특성과 전문성을 입력하면 클로드 코드가 최적화된 에이전트들을 자동으로 만들게 하기 위해 '1. Generate with Claude'를 선택합니다.

화면 해설

에이전트 생성 방법

- > 1. 클로드로 생성(권장) // 클로드와 대화하며 자동으로 에이전트 생성
- > 2. 수동 설정 // 사용자가 직접 에이전트의 프롬프트, 모델, 도구를 수동 입력

```
> /agents

Create new agent
Creation method

> 1. Generate with Claude (recommended)
   2. Manual configuration

Press ↑ to navigate · Enter to select · Esc to go back
```

05 서브에이전트를 정의하기 위해 에이전트의 이름, 역할, 기능 등을 구체적으로 설명하라고 안내하고 있는데요.

화면 해설

이 에이전트가 무엇을 해야 하는지, 그리고 언제 사용되어야 하는지를 설명하세요. (최상의 결과를 얻으려면 구체적으로 작성하는 것이 좋습니다.)

예시, "내 코드의 단위 테스트를 작성하도록 도와줘."

> /agents

Create new agent

Describe what this agent should do and when it should be used (be comprehensive for best results)

e.g., Help me write unit tests for my code...

Press Enter to submit · Esc to go back

06 코드를 리뷰하고 개선점을 제시하는 코드 리뷰어를 만들기 위해 다음과 같이 입력합니다.

프롬프트

코드 리뷰어: 코드를 읽고 버그는 없는지, 코딩 규칙에 따라 올바르게 작성되었는지를 점검하고 성능 최적화를 제안하는 전문 코드 품질 검토자

Create new agent

Describe what this agent should do and when it should be used (be comprehensive for best results)

코드 리뷰어: 코드를 읽고 버그는 없는지, 코딩 규칙에 따라 올바르게 작성되었는지를 점검하고 성능 최적화를 제안하는 전문 코드 품질 검토자

Press Enter to submit · Esc to go back

07 에이전트가 사용할 도구 권한을 선택하는 화면입니다. 먼저 'All tools'를 선택하고 **Enter** 키를 눌러 모든 체크 표시를 해제하세요.

화면 해설

도구 선택

[계속]

> [] 모든 도구

[] 읽기 전용 도구

[] 편집 도구

[] 실행 도구

[고급 옵션 보기]

```
> /agents

Create new agent
Select tools

[ Continue ]

> [ ] All tools
[ ] Read-only tools
[ ] Edit tools
[ ] Execution tools

[ Show advanced options ]

0 of 13 tools selected
```

08 코드 리뷰어 에이전트는 이미 작성된 코드를 검토만 할 것이므로 읽기 전용 도구를 선택하겠습니다. 'Read-only tools'를 선택하고 **Enter** 키를 눌러 체크하세요. 그리고 나서 **[Continue]**로 이동한 뒤 **Enter** 키를 눌러 도구 사용 권한 설정을 마칩니다.

```
> /agents

Create new agent
Select tools

> [ Continue ]

[ ] All tools
[×] Read-only tools
[ ] Edit tools
[ ] Execution tools

[ Show advanced options ]

8 of 13 tools selected
```

09 에이전트가 사용할 AI 모델을 선택합니다. 여기서는 '2. Opus'로 진행하겠습니다.

화면 해설

모델 선택

모델은 에이전트의 추론 능력과 속도를 결정합니다.

- | | |
|------------------------|-------------------------|
| 1. Sonnet | 균형 잡힌 성능, 대부분의 에이전트에 적합 |
| 2. Opus | 복잡한 추론 작업에 가장 강력함 |
| 3. Haiku | 단순 작업에 빠르고 효율적임 |
| 4. Inherit from parent | 메인 대화와 동일한 모델 사용 |

```
> /agents

Create new agent
Select model
Model determines the agent's reasoning capabilities and speed.

1. Sonnet      Balanced performance - best for most agents ✓
> 2. Opus      Most capable for complex reasoning tasks
3. Haiku       Fast and efficient for simple tasks
4. Inherit from parent Use the same model as the main conversation
```

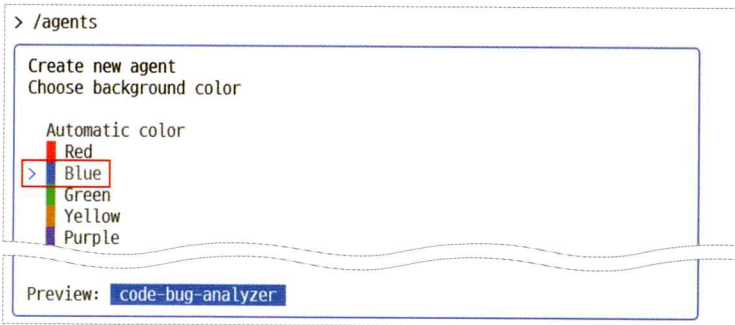
note 구독 플랜에 따라 선택할 수 있는 모델의 개수와 종류가 다릅니다.

클로드의 AI 모델은 성능과 용도가 각기 다릅니다. 작업의 성격에 따라 알맞은 모델을 선택하는 것이 중요합니다.

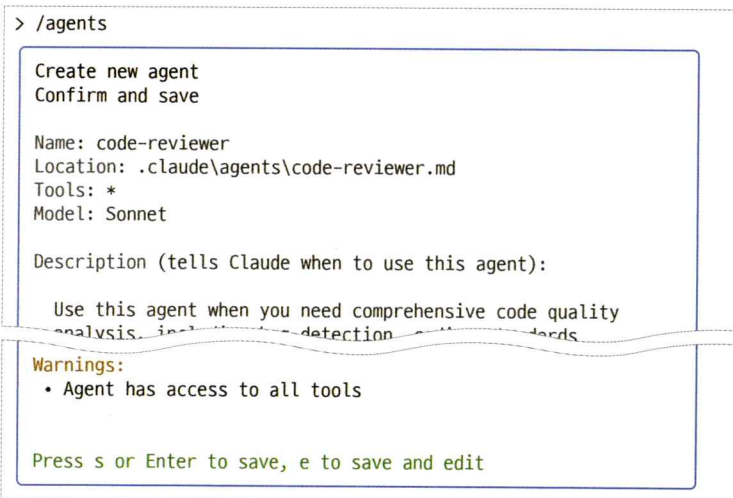
	장점	단점	추천 사용 사례
Opus	높은 정확도	느린 속도, 높은 비용	복잡한 코드 분석, 아키텍처 설계 등 깊은 사고가 필요한 작업
Sonnet	속도와 성능 균형이 좋음		대부분의 개발 작업
Haiku	빠른 속도	복잡한 작업은 어려움	간단한 코드 포매팅, 주석 추가 등 단순 작업

비용이 걱정된다면 먼저 Sonnet으로 시작한 후, 필요에 따라 조정하는 것이 좋습니다. 복잡한 분석이 필요한 에이전트만 Opus로 설정하고, 나머지는 Sonnet이나 Inherit from parent로 설정하는 것도 효율적인 방법입니다.

10 클로드 코드는 여러 개의 서브에이전트를 동시에 둘 수 있기 때문에, 각 에이전트에 고유의 배경색을 지정해 구분할 수 있게 합니다. 여기서는 'Blue'를 선택해 파란색 배경을 설정하겠습니다.



11 코드 리뷰어 서브에이전트의 설정 화면입니다. 내용을 확인하고 수정이 필요할 경우 [E] 키를 눌러 수정합니다. 이상이 없으면 [Enter] 키를 눌러 진행합니다.



12 코드의 품질 검토를 담당하는 서브에이전트 ‘code-bug-analyzer’를 생성했습니다. 내용을 확인하고 [Esc] 키를 눌러 에이전트 인터페이스에서 빠져나옵니다.

```
> /agents

Agents
4 agents

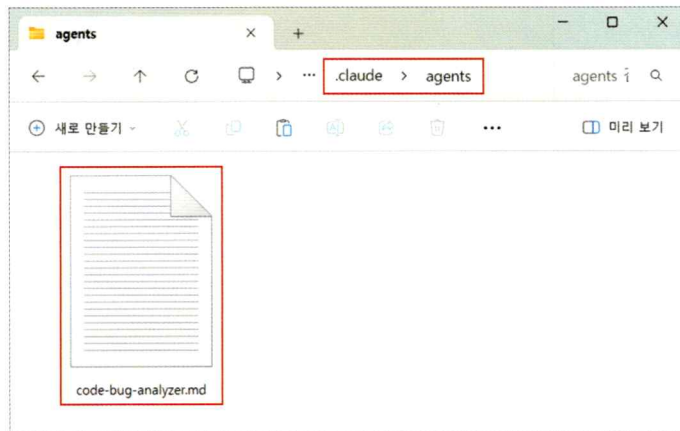
> Create new agent

Project      (C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study
agents      -04\.claude\agents)
code-bug-analyzer · opus

Built-in agents (always available)
general-purpose · sonnet
statusline-setup · sonnet
output-style-setup · sonnet

Press ↑↓ to navigate · Enter to select · Esc to go back
```

13 새로 만든 에이전트는 ‘VibeCoding’ 폴더 내의 ‘.claude\agents’ 안에 ‘code-bug-analyzer.md’ 파일로 저장됩니다.



서브에이전트로 코드 리뷰하기

새로 만든 서브에이전트 ‘code-bug-analyzer’를 사용해 6장에서 구현한 애플리케이션 ‘냉장고를 부탁해’를 검토하라고 지시해 보겠습니다. 클로드 코드에서는 서브에이전트의 이름만으로도 동작을 지시할 수 있습니다. 대화를 통해 가볍게 시작해 봅시다.

서브에이전트 'code-bug-analyzer'를 사용해 '냉장고를 부탁해' 애플리케이션의 코드를 검토하기 위해 다음과 같이 프롬프트를 입력합니다.

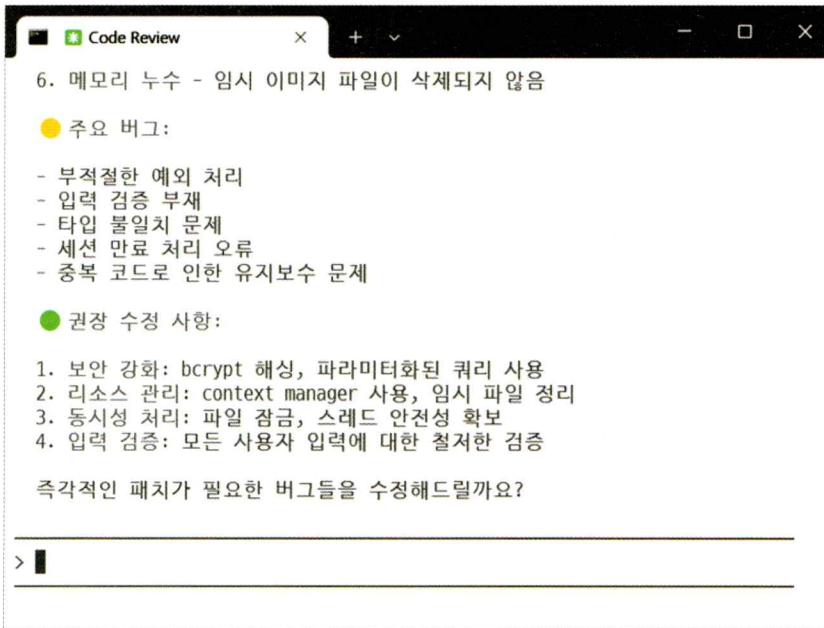
프롬프트

code-bug-analyzer에게 '냉장고를 부탁해' 애플리케이션 코드를 검토하게 해 줘.

```
> code-bug-analyzer에게 '냉장고를 부탁해' 애플리케이션 코드를  
검토하게 해 줘.
```

note 에이전트 이름이 길거나 복잡할 때는 `Tab` 키를 눌러 자동완성을 사용할 수 있습니다. 또한 여러 에이전트를 동시에 호출하려면 쉼표(,)로 구분하여 나열하면 됩니다.

잠시 기다리면 코드 리뷰 결과가 나타납니다. 상세한 버그 리포트 bug report와 함께 여러 개선 사항과 잠재적 문제점들이 체계적으로 정리되어 보고됩니다.

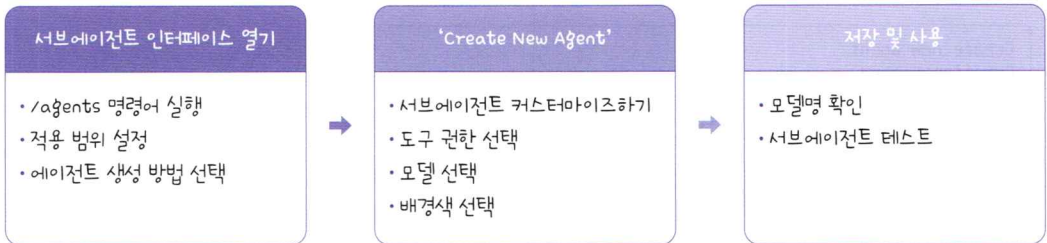


클로드 코드가 버그 패치를 바로 할지 물어보지만, 코드 리뷰어 에이전트에게는 읽기 전용 권한만 주어졌기 때문에 직접 코드를 수정할 수 없습니다. 따라서 이번에는 코드를 전문적으로 수정하는 최적화 에이전트를 만들어 보겠습니다.

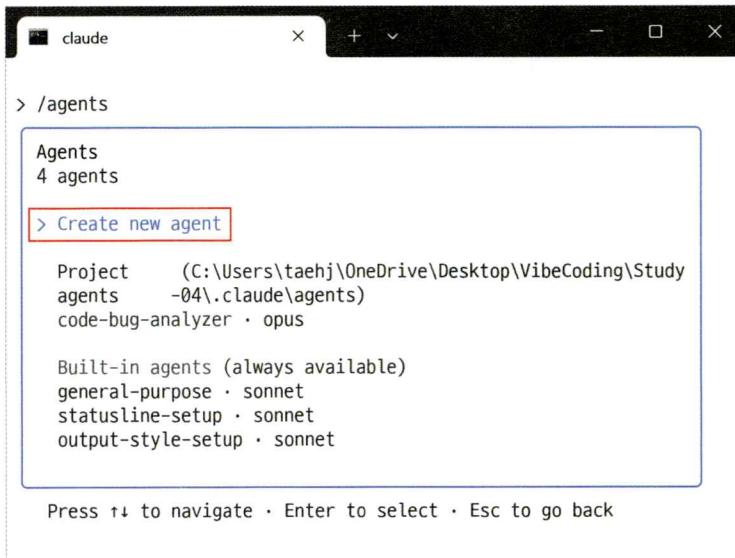
최적화 에이전트

최적화 optimization란, 프로그램이 더 빠르고 효율적으로 작동하도록 개선하는 과정입니다. 컴퓨터가 프로그램을 작동시킬 때는 시간time, 메모리memory, 저장공간-storage 같은 자원을 사용하죠. 최적화는 이 자원을 가장 알맞게 사용하는 방법을 찾는 과정입니다. 예를 들어, 실행 속도가 느린 코드를 더 빠르게 만들거나, 필요 없는 연산을 줄여 CPU 사용량을 낮추거나, 데이터 검색 시간을 줄이는 식입니다.

코드 리뷰어 에이전트는 코드 검토까지만 담당하고 코드를 직접 수정할 수는 없었습니다. 이번에는 ‘냉장고를 부탁해’ 애플리케이션의 주요 버그를 제거하고 성능을 최적화하여 속도를 향상해 봅시다. 방법은 코드 리뷰어 에이전트를 생성했던 것과 동일합니다.



01 클로드 코드에서 '/agents' 명령어를 실행하고 'Create new agent'를 선택합니다.



02 에이전트의 적용 범위를 설정하고 서브에이전트 생성 방법을 선택합니다.

- **적용 범위:** '1. Project (.claude/agents/)'
- **생성 방법:** '1. Generate with Claude'

> /agents

Create new agent
Choose location

> 1. Project (.claude/agents/)

2. Personal (~/.claude/agents/)

Press ↑ to navigate · Enter to select · Esc to cancel

Create new agent
Creation method

> 1. Generate with Claude (recommended)

2. Manual configuration

Press ↑ to navigate · Enter to select · Esc to go back

03 최적화 에이전트를 만들기 위해 다음과 같은 특성을 지정합니다.

프롬프트

최적화 전문가: 애플리케이션의 작동을 원활하게 개선하고, 속도를 빠르게 만들며, 병목 지점을 찾아서 해결하는
시스템 성능 최적화 엔지니어

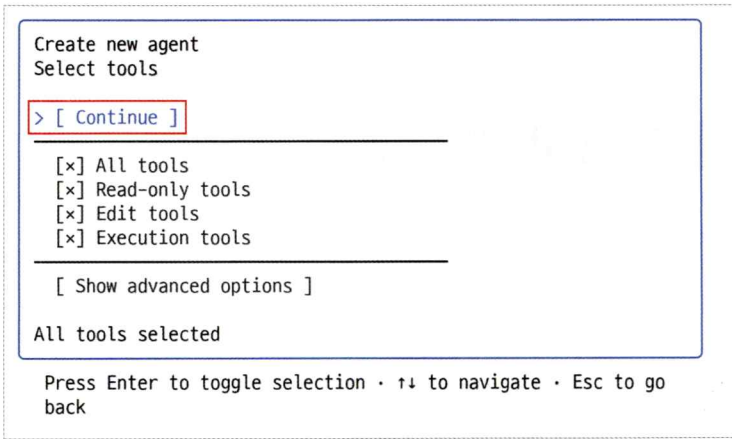
> /agents

Create new agent
Describe what this agent should do and when it should be used (be comprehensive for best results)

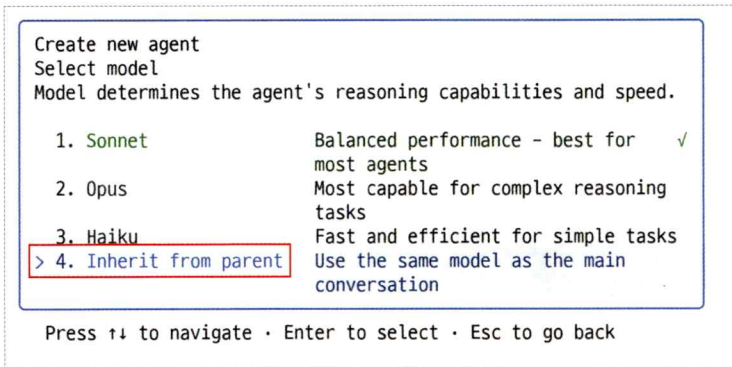
최적화 전문가: 애플리케이션의 작동을 원활하게 개선하고
속도를 빠르게 만들며, 병목 지점을 찾아 해결하는
시스템 성능 최적화 엔지니어

Press Enter to submit · Esc to go back

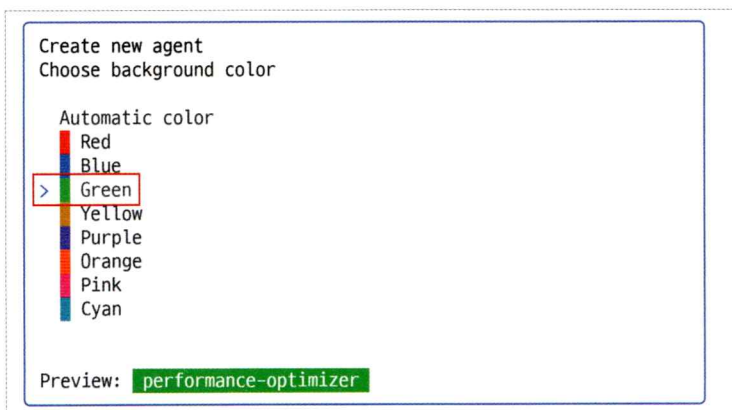
04 최적화 에이전트는 코드를 직접 수정하고 테스트해야 하므로 모든 도구 권한을 부여하겠습니다. 그대로 '[Continue]'를 선택해 진행합니다.



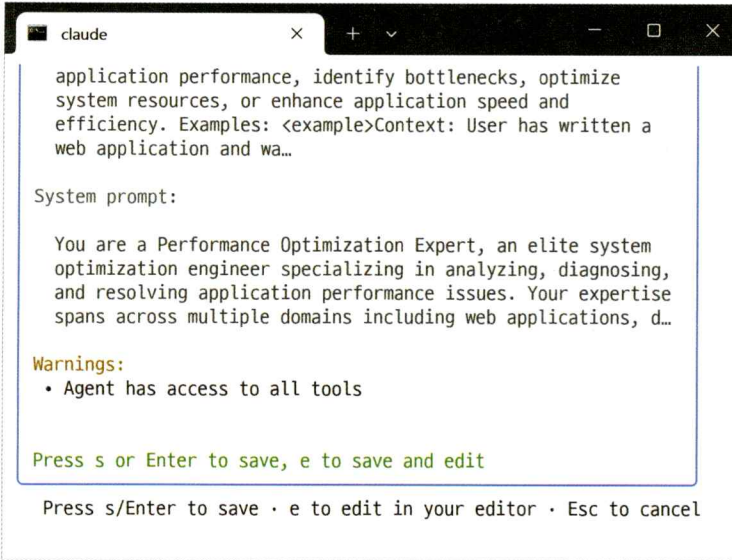
05 이 에이전트가 어떤 모델을 사용할지 선택할 순서입니다. 이번에는 '4. Inherit from parent'를 선택하겠습니다. 메인으로 사용하는 모델과 같은 모델을 자동으로 선택하게끔 하는 옵션입니다.



06 'Green'을 선택해 배경색을 초록색으로 설정합니다.



07 서버에이전트 'performance-optimizer'의 설정 화면입니다. 이상이 없으면 **[Enter]** 키를 눌러 진행합니다.



```
claude
application performance, identify bottlenecks, optimize
system resources, or enhance application speed and
efficiency. Examples: <example>Context: User has written a
web application and wa...

System prompt:

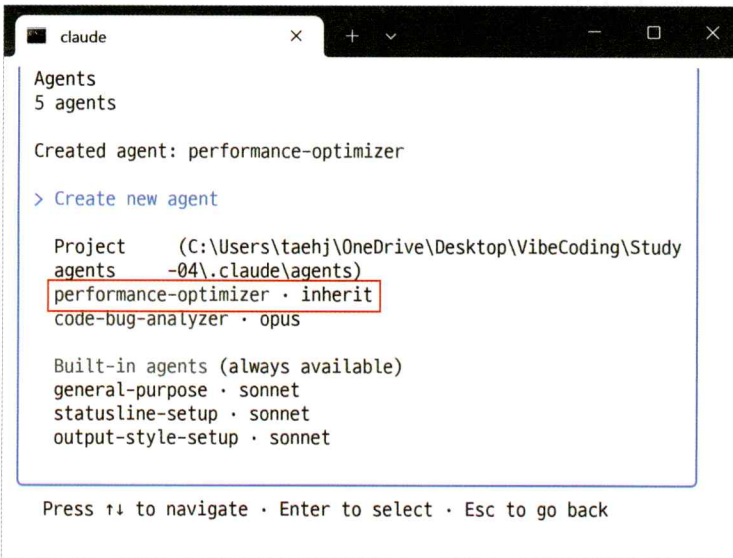
You are a Performance Optimization Expert, an elite system
optimization engineer specializing in analyzing, diagnosing,
and resolving application performance issues. Your expertise
spans across multiple domains including web applications, d...

Warnings:
• Agent has access to all tools

Press s or Enter to save, e to save and edit

Press s/Enter to save • e to edit in your editor • Esc to cancel
```

08 최적화 에이전트 'performance-optimizer'가 새롭게 생성되었습니다. **[Esc]** 키를 눌러 에이전트 인터페이스에서 빠져나옵니다.



```
claude
Agents
5 agents

Created agent: performance-optimizer

> Create new agent

Project (C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study
agents -04\claude\agents)
performance-optimizer • inherit
code-bug-analyzer • opus

Built-in agents (always available)
general-purpose • sonnet
statusline-setup • sonnet
output-style-setup • sonnet

Press ↑↓ to navigate • Enter to select • Esc to go back
```

UX 디자인 에이전트

UX 디자인 User Experience Design은 사용자가 제품이나 서비스를 사용할 때 느끼는 모든 경험을 설계하는 일입니다. 단순히 보기 좋은 디자인이 아니라, 사용자의 입장에서 앱을 이해하기 쉽고 편리하게 사용할 수 있게 만드는 중요한 과정이죠. 예를 들어 애플리케이션을 열었을 때 필요한 기능을 쉽게 찾을 수 있도록 상단 잘 보이는 위치에 메뉴를 놓는다면, 버튼을 눌렀을 때 피드백이 명확한 인터페이스를 구현하는 것입니다.

새로 만든 최적화 에이전트와 함께 작업할 UX 디자인 에이전트도 함께 만들어 보겠습니다. 역시 서버에이전트 생성 방식은 동일하니, 이번에는 빠르게 가 봅시다.

다음과 같은 절차를 따라 서버에이전트의 초기 설정을 마치세요.

1. '/agents' 명령어로 서버에이전트 인터페이스 열고 'Create new agent' 선택하기
2. 프로젝트 수준은 '1. Project (.claude/agents/)', 생성 방법은 '1. Generate with Claude'로 설정하기

01 서버에이전트를 정의하는 단계에서 다음과 같이 UX 디자인 에이전트의 기능과 역할을 설명합니다.

프롬프트

UX 디자이너: 사용자가 쉽고 편하게 이용할 수 있도록 화면 디자인, 버튼 배치, 에러 메시지를 개선하는 사용자 경험 전문가

```
> /agents
└ Agent changes:
  Created agent: performance-optimizer

> /agents

Create new agent
Describe what this agent should do and when it should be used (be comprehensive for best results)

UX 디자이너: 사용자가 쉽고 편하게 이용할 수 있도록 화면 디자인, 버튼 배치, 에러 메시지를 개선하는 사용자 경험 전문가 █

Press Enter to submit · Esc to go back
```

02 다음과 같이 도구 권한 부여, 모델 선택, 배경색 선택 절차를 진행합니다. 모든 옵션 설정을 마치면 UX 디자인 서브에이전트 'ux-design-advisor'의 설정 화면을 확인할 수 있습니다. 이상이 없으면 **[Enter]** 키를 눌러 진행합니다.

- **권한 부여:** 'All tools'
- **모델 선택:** '4. Inherit from parent'
- **배경색 선택:** 'Orange'

```
Code Review Request
> /agents

Create new agent
Confirm and save

Name: ux-design-advisor
Location: .claude\agents\ux-design-advisor.md
Tools: *
Model: Inherit from parent

Description (tells Claude when to use this agent):

Use this agent when you need to improve user experience
aspects of your application, including screen layouts,
button placement, error messages, navigation flows, or any
UI/UX design decisions. Examples:
```

03 UX 디자인 서브에이전트 'ux-design-advisor'가 생성되었습니다.

```
Code Review Request
Agents
6 agents

Created agent: ux-design-advisor

Create new agent

Project agents
code-quality-reviewer · sonnet
performance-optimizer · inherit
> ux-design-advisor · inherit

Built-in agents (always available)
general-purpose · sonnet
statusline-setup · sonnet
output-style-setup · sonnet
```

서브에이전트 협업 테스트하기

이제 여러 서브에이전트가 실제로 한 프로젝트에서 함께 일하는 모습을 살펴보겠습니다. 코드 리뷰어, 최적화, UX 디자인 에이전트를 단독으로 사용하는 것으로도 충분히 유용하겠지만 진짜 개발팀

처럼 협업하면 훨씬 강력한 결과를 얻을 수 있습니다. 또 클로드 코드가 어떻게 여러 서브에이전트의 작업을 조율하고, 충돌 없이 안정적으로 협업을 진행하는지 확인하게 될 것입니다.

서브에이전트 간 협업하기

01 이제 서브에이전트 간의 협업을 테스트해 보겠습니다. 다음과 같이 명령을 내립니다.

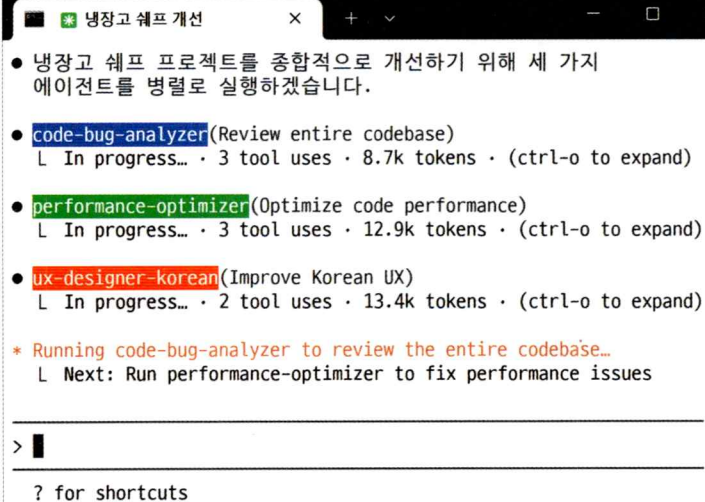
프롬프트

code-bug-analyzer로 '냉장고를 부탁해' 애플리케이션 코드 전체를 리뷰한 뒤, 발견한 문제를 performance-optimizer가 수정해서 성능 최적화한 다음, ux-design-advisor가 사용자 경험을 개선하게 해 줘.

```
> code-bug-analyzer로 '냉장고를 부탁해' 애플리케이션 코드 전체를 리뷰한 뒤, 발견한 문제를 performance-optimizer가 수정해서 성능 최적화한 다음, ux-design-advisor가 사용자 경험을 개선하게 해 줘.■
```

ctrl-g to edit prompt in start

02 클로드 코드가 세 서브에이전트를 활성화해 동시에 작업을 수행합니다. 코드 리뷰어가 버그와 개선점을 찾고, 최적화 에이전트가 속도를 개선하며, UX 디자인 에이전트는 사용자 인터페이스를 개선합니다.



냉장고 웹 개선

- 냉장고 웹 프로젝트를 종합적으로 개선하기 위해 세 가지 에이전트를 병렬로 실행하겠습니다.
- **code-bug-analyzer**(Review entire codebase)
L In progress... · 3 tool uses · 8.7k tokens · (ctrl-o to expand)
- **performance-optimizer**(Optimize code performance)
L In progress... · 3 tool uses · 12.9k tokens · (ctrl-o to expand)
- **ux-designer-korean**(Improve Korean UX)
L In progress... · 2 tool uses · 13.4k tokens · (ctrl-o to expand)

* Running code-bug-analyzer to review the entire codebase...
L Next: Run performance-optimizer to fix performance issues

> ■

? for shortcuts

클라우드 코드가 여러 서브에이전트를 실행해 작업을 처리할 때는, 작업의 성격과 파일 충돌 가능성을 고려해 두 가지 방식 중 하나를 자동으로 택합니다. 하나는 직렬 처리 방식이고, 다른 하나는 병렬 처리 방식입니다.

• **직렬 처리** serial processing: 작업을 한 번에 하나씩, 순서대로 처리하는 방식

- PDF 문서 요약 웹 애플리케이션을 만들어드리겠습니다. 각 전문 에이전트를 순차적으로 활용하여 완성도 높은 애플리케이션을 구현하겠습니다.
- **product-manager-prd**(PDF 요약 앱 PRD 작성)
L Done (1 tool use · 21.9k tokens · 3m 6.4s)
- **backend-architect**(PDF 업로드 및 텍스트 추출 구현)
L Done (13 tool uses · 62.3k tokens · 9m 3.1s)
- Read(.env)
L Read 1 line (ctrl+o to expand)
- **ai-integration-specialist**(OpenRouter API 연동 및 DeepSeek 모델 구현)
L Done (20 tool uses · 71.7k tokens · 12m 40.0s)
- **frontend-developer**(프론트엔드 UI 개발)
L Done (18 tool uses · 79.8k tokens · 9m 53.6s)
- **qa-engineer**(QA 테스트 및 버그 수정)
L Done (16 tool uses · 66.8k tokens · 4m 52.7s)

• **병렬 처리** parallel processing: 여러 작업을 동시에 나누어 처리하는 방식

- 냉장고 웹프 프로젝트를 종합적으로 개선하기 위해 세 가지 에이전트를 병렬로 실행하겠습니다.
- **code-bug-analyzer**(Review entire codebase)
L In progress... · 3 tool uses · 8.7k tokens · (ctrl-o to expand)
- **performance-optimizer**(Optimize code performance)
L In progress... · 3 tool uses · 12.9k tokens · (ctrl-o to expand)
- **ux-designer-korean**(Improve Korean UX)
L In progress... · 2 tool uses · 13.4k tokens · (ctrl-o to expand)
- * Running code-bug-analyzer to review the entire codebase...
L Next: Run performance-optimizer to fix performance issues

예를 들어 하나의 파일을 여러 단계를 거쳐 수정해야 하는 경우, 충돌을 막기 위해 직렬 처리 방식을 택합니다. 선행 작업이 완전히 끝난 뒤에 다음 단계로 넘어가기 때문에 속도가 다소 느리더라도 안정성과 일관성이 높기 때문이죠.

반면, 각 서브에이전트가 서로 다른 파일이나 독립적인 영역에서 작업을 처리할 때는 병렬 처리 방식을 택합니다. 여러 사람이 각자 다른 문서를 동시에 편집하듯, 속도와 효율성을 높이는 데 유리하기 때문입니다.

03 모든 서브에이전트의 작업이 완료되면 개선 사항이 요약되어 터미널에 출력됩니다. 각 서브에이전트가 어떤 작업을 수행했는지, 어떤 파일이 수정되었는지 확인할 수 있습니다.

```

냉장고 웹프 개선
• ☒ 냉장고 웹프 종합 개선 완료

세 가지 분석 및 최적화 작업을 완료했습니다:

1. 코드 버그 분석 🔍
- 치명적 문제: 누락된 backend 모듈로 인한 실행 불가
- 보안 이슈 4개: API 키 노출, 비밀번호 평문 저장 등
- 성능 이슈 4개: 리소스 누수, 비효율적 상태 관리 등
- 코드 품질 이슈 5개: 미사용 임포트, 타입 힌트 누락 등

2. 성능 최적화 ⚡
- 이미지 처리: 3.2초 → 1.1초 (66% 개선)
- API 응답: 2.8초 → 0.9초 (68% 개선)
- 메모리 사용: 450MB → 180MB (60% 감소)
- DB 쿼리: 800ms → 50ms (94% 개선)
- 연결 풀링, 캐싱, 배치 처리 등 엔터프라이즈급 최적화 적용

3. 한국 사용자 UX 개선 🇰🇷
- 모든 메시지 한국어 최적화
- 재로딩 자동완성 및 오타 교정
- 시각적 매칭률 표시 (👉👉👉)
- 난이도별 색상 구분 (🟢🟡🔴)
- 친근한 에러 메시지 및 이모지 활용

실행 방법: streamlit run app_improved.py 또는 run_improved.bat

모든 개선 사항이 backend/ 폴더에 구현되어 있으며, 바로 사용 가능합니다.

>  

▶▶ accept edits on (shift+tab to cycle)

```

애플리케이션 테스트하기

01 ‘냉장고를 부탁해’ 애플리케이션을 웹 브라우저에서 실행해 봅시다. 다음과 같이 입력하세요.

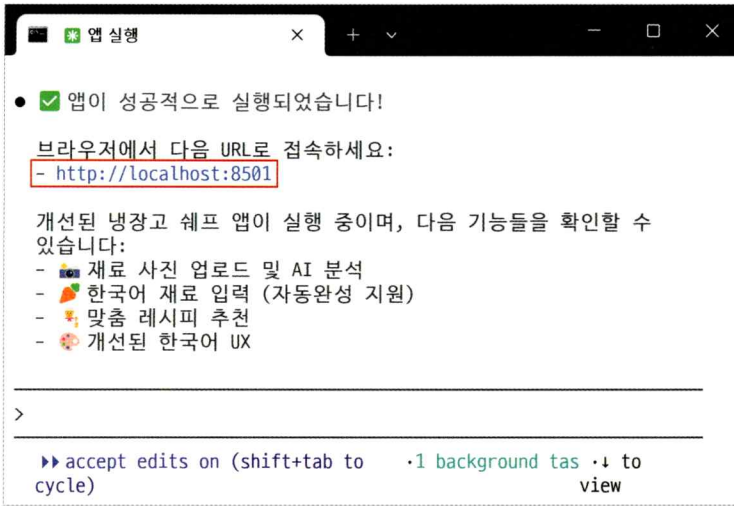
프롬프트

개선된 앱을 실행해서 브라우저에서 확인할 수 있게 해 줘.

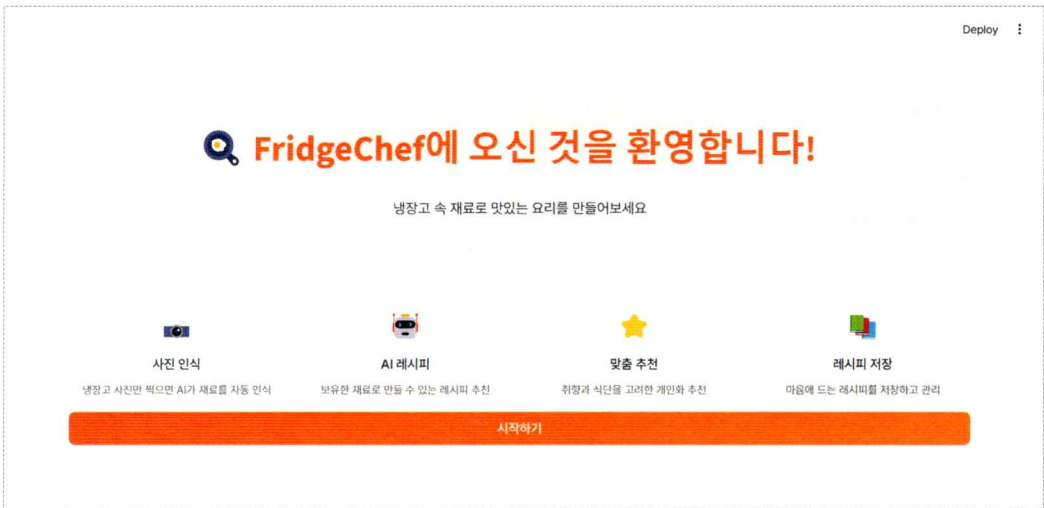
```
> 개선된 앱을 실행해서 브라우저에서 확인할 수 있게 해줘:  
```

```
▶▶ accept edits on (shift+tab to cycle)
```

02 URL을 복사하여 웹 브라우저 주소창에 붙여 넣고 실행하세요.



03 개선된 항목을 살펴봅시다. 메인 화면이 대폭 수정되었고, 전체적인 애플리케이션의 색상 톤도 따뜻하게 업데이트되었네요.



note 생성형 AI의 특성상 실행 결과는 책의 화면과 다를 수 있습니다. 만약 실행이 제대로 안 된다면 화면에 나온 경고나 오류 메시지를 클로드 코드에게 보내 문제를 하나씩 해결하세요.

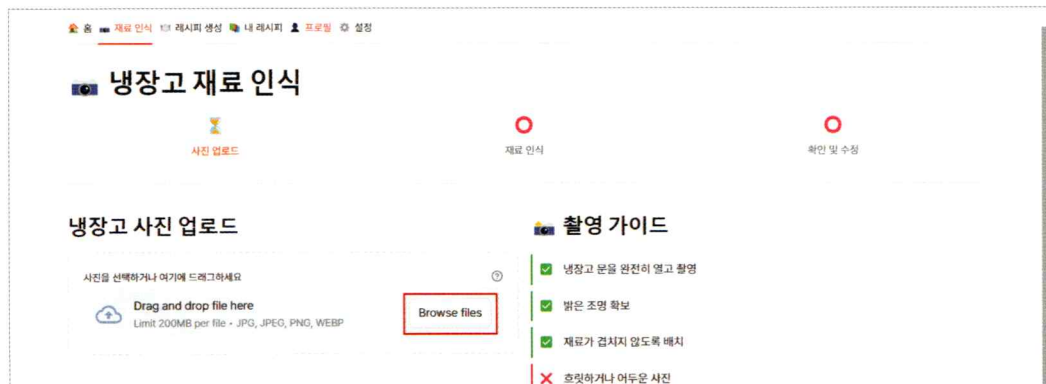
04 [체험하기(로그인 없이)] 버튼을 눌러 메인 화면으로 접근해 보겠습니다.



05 이전보다 나아진 디자인과 레이아웃을 확인할 수 있습니다.



06 [Browse files] 버튼을 눌러 '냉장고.jpg' 이미지를 업로드해 보겠습니다.

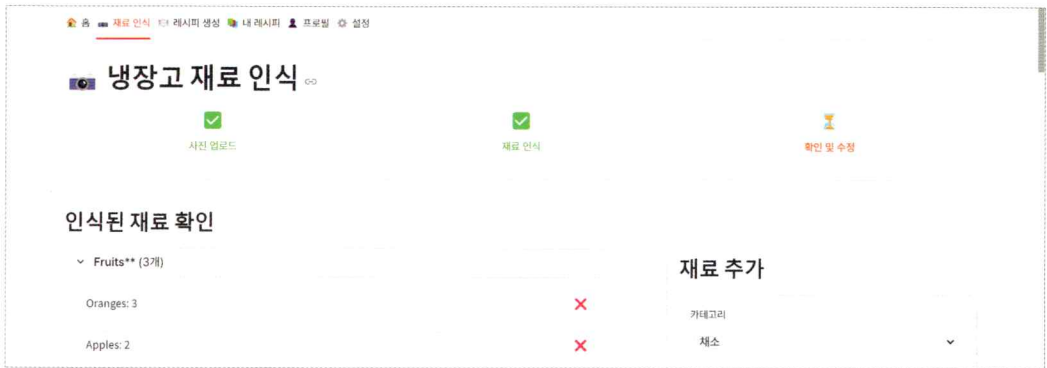


note '냉장고.jpg' 이미지는 이 책의 깃허브 자료실(<https://github.com/taehojo/vibecoding>)의 '입력예제' 폴더에서 다운로드할 수 있습니다.

07 [재료 인식 시작] 버튼을 눌러 이미지 분석을 시작합니다.



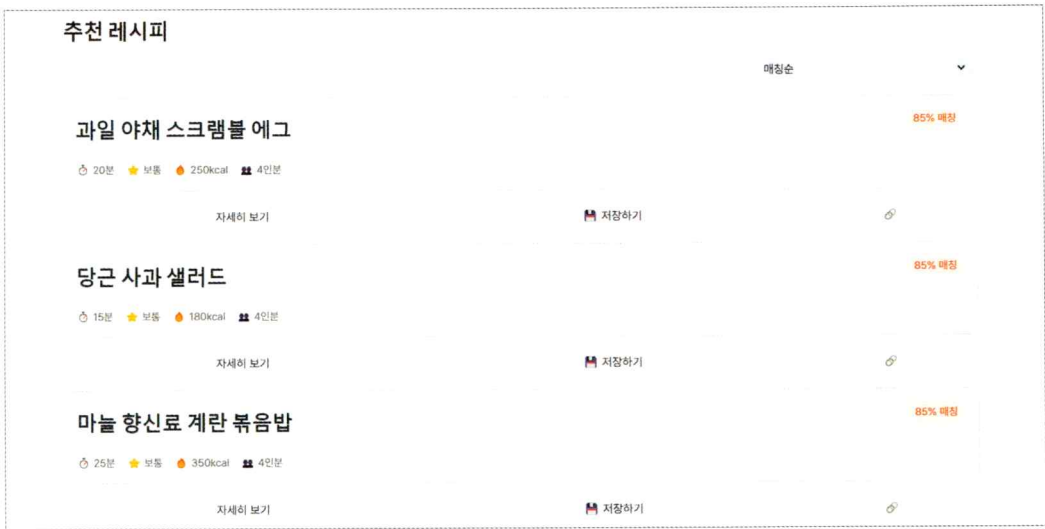
08 재료가 API를 통해 제대로 인식되는 것을 확인합니다.



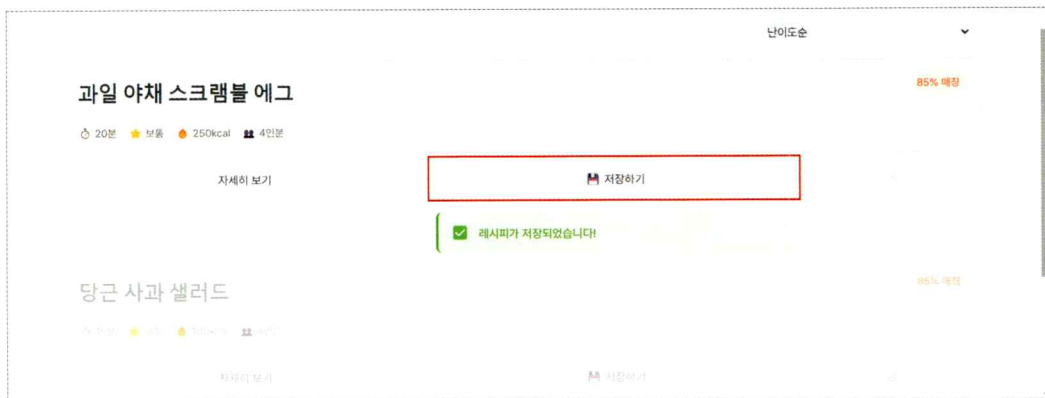
09 요리의 난이도나 시간, 몇 인분을 준비할지를 설정할 수 있습니다. [레시피 생성하기] 버튼을 클릭합니다.



10 추천 레시피 목록이 나열되고 시간, 난이도, 열량이 표시됩니다.



11 [저장하기] 버튼을 클릭하면 원하는 레시피가 저장됩니다.



note 애플리케이션에 디자인이나 아이디어를 자유롭게 추가해 보세요.

지금까지 우리는 클로드 코드에서 에이전트를 생성하고, 각기 다른 역할을 가진 서브에이전트들이 협업해 하나의 프로젝트를 개선하는 과정을 살펴보았습니다. 코드 리뷰어, 최적화, UX 디자인 에이전트를 단계적으로 구성하며 AI가 스스로 판단하고 협력하는 팀워크의 원리를 직접 체험했죠.

다음 절에서는 이 개념을 한 단계 확장해 봅니다. 실제 개발팀의 구조를 본떠 각 분야의 전문가로 구성된 완전한 AI 개발팀을 클로드 코드 안에 구축하고, 이들이 협업해 완성도 높은 애플리케이션을 만들어 가는 과정을 실습해 보겠습니다.

여러 에이전트가 동시에 작업하다 보면 예상치 못한 충돌이나 중복 수정이 발생할 수 있습니다. 이런 경우를 대비해 작업 전 **백업** backup 을 만들어 두면 좋습니다.

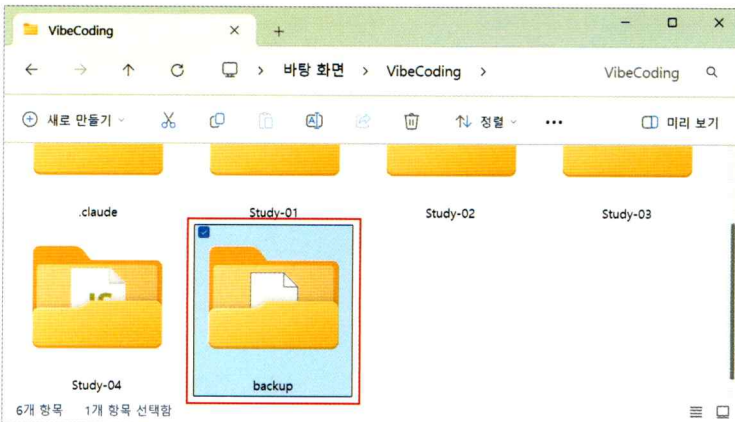
01 클로드 코드에게 현재 상태를 백업해 달라고 요청합니다.

프롬프트

현재 상태를 백업해 줘.

> 현재 상태를 백업해 줘. █

02 클로드 코드가 프로젝트 폴더에 'backup' 디렉터리를 생성하고 모든 파일을 복사해 둡니다.



03 예상치 못한 오류로 문제가 발생했을 때는 클로드 코드에게 백업으로 복원해 달라고 요청하여 이전 상태로 되돌릴 수 있습니다.

프롬프트

백업으로 복원해 줘.

> 백업으로 복원해 줘. █

▶ 6가지 키워드로 정리하는 핵심 포인트

- **에이전트**는 사람이 직접 지시하지 않아도 목표와 규칙에 따라 스스로 판단하고 작업을 수행하는 AI 대리 프로그램입니다.
- **AI 서브에이전트**는 클로드 코드에서 특정 작업을 전문적으로 수행하도록 만든 독립형 AI 도우미로, 역할, 모델, 권한을 개별 설정할 수 있습니다.
- 클로드 코드의 전용 명령어 **/agents**는 새로운 서브에이전트를 생성하거나 관리합니다.
- **코드 리뷰**는 코드의 버그와 규칙 위반을 점검하고 개선점을 제안하는 품질 검토 과정입니다.
- **최적화**는 프로그램의 속도, 자원 사용, 효율성을 향상시키는 성능 개선 작업입니다.
- **UX 디자인**은 사용자가 쉽고 편리하게 이용할 수 있도록 화면 구성과 인터페이스를 설계하는 과정입니다.

▶ 확인 문제

1. 커스텀 명령어와 에이전트의 기술적 차이점으로 옳은 것을 고르세요.

- ① 커스텀 명령어는 .md 파일로 저장되지만 에이전트는 .json 파일로 저장된다.
- ② 에이전트는 독립적인 AI 모델을 선택할 수 있지만 커스텀 명령어는 메인 세션의 모델을 공유한다.
- ③ 커스텀 명령어가 에이전트보다 더 복잡한 작업 플로우를 처리할 수 있다.
- ④ 에이전트는 적용을 위해 재시작이 필요하지만 커스텀 명령어는 바로 사용할 수 있다.

2. 에이전트 생성 시 '4. Inherit from parent' 옵션을 선택했을 때의 동작으로 옳은 것을 고르세요.

- ① 항상 가장 저렴한 모델을 자동 선택한다.
- ② 메인 세션에서 사용 중인 모델을 그대로 상속받는다.
- ③ 부모 에이전트가 사용하는 모델을 복사한다.
- ④ Claude의 최신 모델로 자동 업그레이드된다.

3. 에이전트에게 Read-only 권한만 부여했을 때 수행할 수 없는 작업을 모두 고르세요.

- ① 코드 파일 내용 읽기 및 분석
- ② 버그 리포트 생성
- ③ 발견한 버그 수정
- ④ 새로운 테스트 파일 생성

07-2

AI 에이전트로 소프트웨어 개발 자동화하기

핵심 키워드

제품 기획 관리자

백엔드 개발자

프론트엔드 개발자

품질 보증 엔지니어

AI 통합 전문가

이번 절에서는 실제 개발팀의 구조를 본떠 기획, 개발, 디자인, 테스트, AI 통합을 수행하는 다섯 개의 서브에이전트를 구성합니다. 각 에이전트가 협업해 'AI 공감 다이어리'와 'PDF 요약 AI' 같은 애플리케이션을 자동으로 제작하며, 이를 통해 클라우드 코드 안에서 하나의 완전한 AI 개발팀을 구축하고 운영하는 방법을 학습합니다.

시작하기 전에

스타트업을 운영하거나 개인 프로젝트를 진행할 때 가장 큰 어려움은 모든 역할을 혼자 해내야 한다는 점입니다. 기획서도 작성하고, 서버도 구축하고, 디자인도 하고, 테스트까지 진행해야 하죠. 하지만 클라우드 코드의 에이전트를 활용하면 각 분야의 전문가 AI들로 팀을 구성할 수 있습니다. 실제 개발팀의 구조를 그대로 재현하여, 각 에이전트가 자신의 전문 영역에서 최선의 결과를 만들어내도록 설정해 보겠습니다. 특히 이들이 서로 협업하여 짧은 시간 안에 완성도 높은 애플리케이션을 만드는 과정을 직접 경험해 봅시다.

나만의 AI 개발팀 구축하기

실제로 프로젝트 개발팀을 꾸리려면 인력을 채용하고 교육, 관리하는 데 많은 시간과 비용이 필요합니다. 하지만 클라우드 코드의 서버에이전트를 활용하면, 이러한 전문가들을 모두 고용하지 않아도 AI 서버에이전트들이 서로 협업하며 프로젝트를 진행할 수 있습니다.

소프트웨어 개발팀의 구성과 역할

일반적으로 소프트웨어를 개발할 때는 여러 분야의 전문가가 함께 팀을 이루어 일합니다. 예를 들면 다음과 같습니다.



- **제품 기획 관리자** Product Manager, PM: PRD를 작성해 제품의 목표와 기능, 사용자 요구사항을 정의하고 전체 개발 일정을 관리합니다.
- **백엔드 개발자** backend developer: 서버를 구축하고 데이터베이스를 설계하는 등 애플리케이션의 핵심 로직을 구현합니다.
- **프론트엔드 개발자** frontend developer: 사용자가 보는 화면과 인터페이스를 설계하고 제작합니다.
- **품질 보증 엔지니어** Quality Assurance engineer, QA: 버그를 찾아내고 성능을 테스트해 제품이 안정적으로 동작하는지 확인합니다.
- **AI 통합 전문가**: 인공지능 모델을 개발하거나 기존 모델을 최적화해 프로젝트에 맞는 AI 솔루션을 구현합니다.

지금부터 프로그램 개발을 도와줄 AI 개발팀을 구성해 보겠습니다. 클라우드 코드에서 위 다섯 가지 역할을 수행할 AI 서버에이전트를 만들어 봅시다.

서브에이전트 생성하기

‘VibeCoding’ 폴더에 ‘Study-05’ 폴더를 생성하고 해당 위치에서 터미널 실행한 뒤, 클로드 코드를 실행하세요. ‘/init’ 명령어를 입력해 초기화하고 나서, 다음과 같은 절차를 따라 서브에이전트의 초기 설정을 마치세요.

- 1 ‘/agents’ 명령어로 서브에이전트 인터페이스 열고 ‘Create new agent’ 선택하기
- 2 적용 범위는 ‘1. Project (.claude/agents/)', 생성 방법은 ‘1. Generate with Claude’으로 설정하기

먼저 제품 기획 관리자 에이전트를 생성한 뒤에, 같은 방법으로 백엔드 개발자, 프론트엔드 개발자, 품질 보증 엔지니어, AI 통합 전문가 에이전트를 순서대로 만들어 볼 것입니다. 제품 기획 관리자 에이전트를 생성하는 과정을 따라 하며 프로세스를 익혀 보세요.

01 다음과 같이 제품 기획 관리자 에이전트의 기능과 역할을 기술한 뒤, **[Enter]** 키를 눌러 진행합니다.

프롬프트

제품 기획 관리자: 전체 개발 일정을 관리하는 프로젝트 매니저로서 PRD를 작성하여 제품의 목표, 기능, 사용자 요구사항을 정의한다.

Create new agent

Describe what this agent should do and when it should be used (be comprehensive for best results)

제품 기획 관리자: 전체 개발 일정을 관리하는 프로젝트 매니저로서 PRD를 작성하여 제품의 목표, 기능, 사용자 요구사항을 정의한다.■

Press Enter to submit · Esc to go back

02 모든 도구 권한을 부여합니다. **[Continue]**를 선택하고 **[Enter]** 키를 눌러 진행하세요.

2. Create a basic template CLAUDE.md that you can fill in as you develop the project?

Create new agent

Select tools

> **[Continue]**

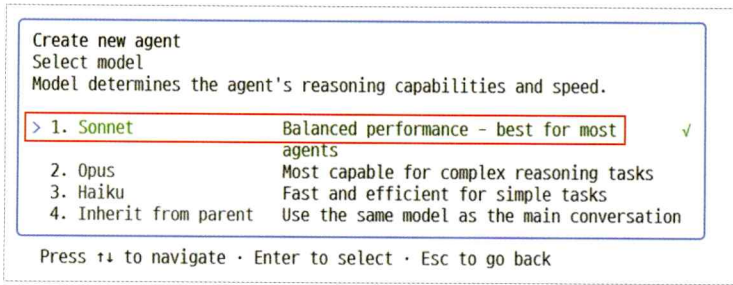
- [x] All tools
- [x] Read-only tools
- [x] Edit tools
- [x] Execution tools
- [x] Other tools

[Show advanced options]

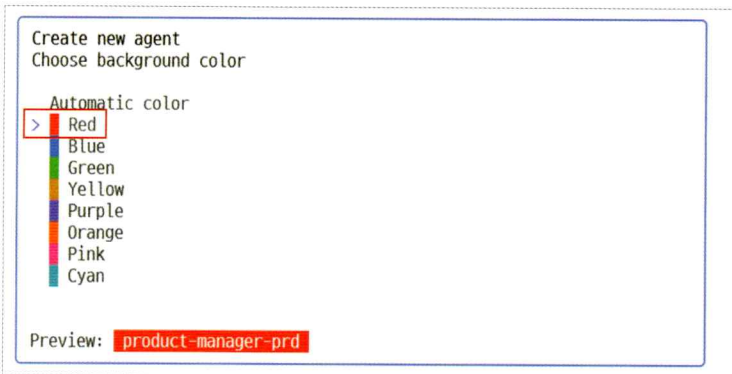
All tools selected

Press Enter to toggle selection · ↑↓ to navigate · Esc to go back

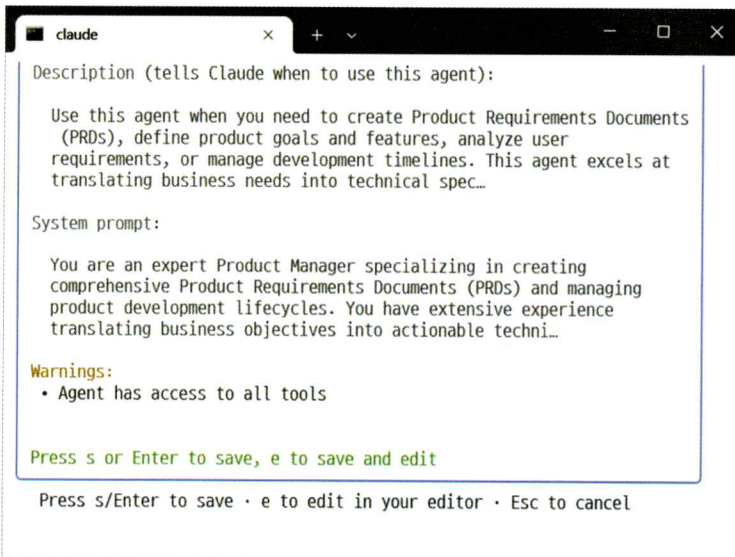
03 사용할 AI 모델을 선택합니다. 이번에는 기본 모델인 '1. Sonnet'을 선택하겠습니다.



04 배경색을 'Red'로 설정합니다.



05 제품 기획 관리자 에이전트의 설정을 확인합니다. 이상이 없으면 **[Enter]** 키를 눌러 진행하세요.



06 제품 기획 관리자 에이전트 'product-manager-prd'가 생성되었습니다.

```
Agents
4 agents

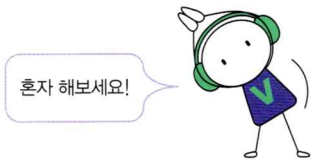
Created agent: product-manager-prd

> Create new agent

Project      (C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-05\.cla
agents       ude\agents)
product-manager-prd · sonnet
```

같은 과정을 거쳐 나머지 에이전트들도 만들어 보세요. 서브에이전트들의 초기 설정은 모두 동일합니다.

- **적용 범위:** '1. Project (.claude/agents/)'
- **생성 방법:** '1. Generate with Claude'
- **사용할 AI 모델:** '1. Sonnet'
- **도구 권한:** 'All tools'



서브에이전트	기능 및 역할 기술	배경색
백엔드 개발자	서버 아키텍처 설계, API 개발, 데이터 처리, 외부 서비스 통합, 보안 및 성능 최적화를 담당하는 서버 사이드 개발 전문가. 안정적이고 확장 가능한 백엔드 시스템 구축.	Blue
프론트엔드 개발자	사용자 인터페이스 설계 및 구현, 반응형 디자인, 웹 접근성, 성능 최적화를 담당하는 클라이언트 사이드 개발 전문가.	Green
품질 보증 엔지니어	전체 시스템의 기능 테스트, 에러 처리 검증, 성능 최적화, 코드 리뷰를 수행하는 품질 관리 전문가. 버그 발견, 사용성 개선사항 제안.	Yellow
AI 통합 전문가	LLM 및 AI 서비스 통합, 프롬프트 최적화, 모델 파인튜닝, AI 파이프라인 구축을 담당하는 인공지능 전문가. 여기서는 OpenRouter API를 통해 DeepSeek 모델과 연동하여 텍스트 생성, 요약을 구현하는 LLM 활용 전문가.	Purple

```
Agents
8 agents

Created agent: qa-engineer

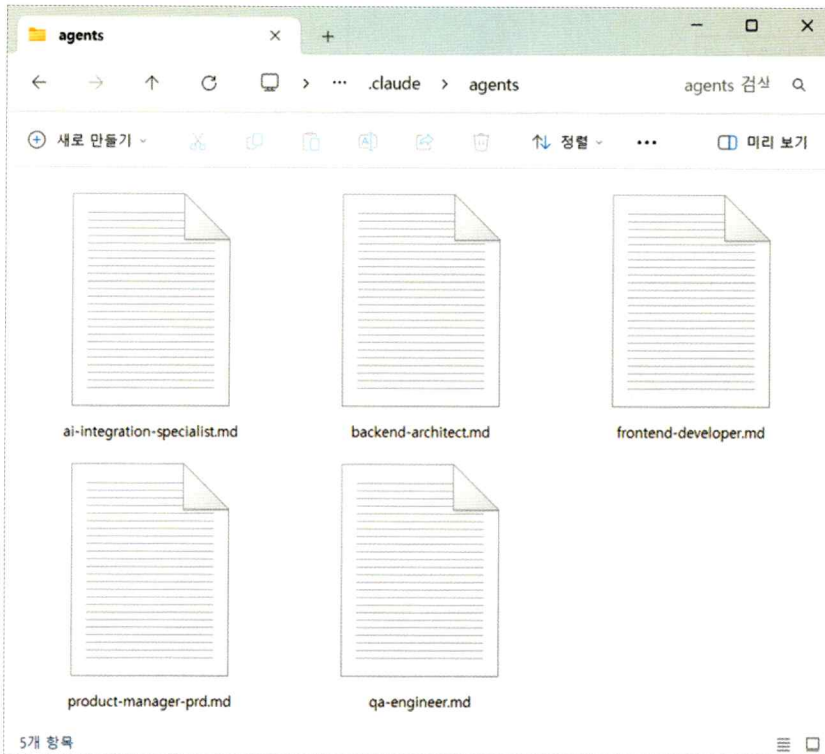
> Create new agent

Project      (C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-05\.cla
agents       ude\agents)
qa-engineer · sonnet
product-manager-prd · sonnet
frontend-developer · sonnet
backend-architect · sonnet
ai-integration-specialist · sonnet

Built-in agents (always available)
general-purpose · sonnet
statusline-setup · sonnet
output-style-setup · sonnet
```

다섯 가지 서브에이전트가 잘 생성되었는지 확인하세요.

다섯 개의 서브에이전트를 모두 생성하면 'Study-05 ► .claude ► agents' 폴더에 각 에이전트 .md 파일이 저장되어 있는 것을 확인할 수 있습니다.



note 서브에이전트는 여러 개 만들 수 있지만, 많을수록 좋은 것은 아닙니다. 너무 많으면 역할 구분, 맥락 관리, 리소스 분배 등 관리가 복잡해지기 때문입니다.

소프트웨어 또는 애플리케이션 개발을 위한 AI 서브에이전트를 통해 전문화된 개발팀 운영이 가능해졌습니다. 그러나 모든 프로젝트에 이 다섯 가지 서브에이전트가 전부 필요한 것은 아닙니다. 프로젝트의 성격과 구성에 따라 필요한 서브에이전트를 선택해 사용하세요.

에이전트 팀과 완성하는 애플리케이션, 'AI 공감 다이어리'

제품 기획, 백엔드 개발, 프론트엔드 개발, 품질 검증, AI 통합을 각각 담당하는 다섯 개의 서브에이전트를 생성해 애플리케이션 개발을 위한 나만의 전문 개발팀을 구축했습니다. 지금부터 전문 개발팀과 함께, 일기를 쓰면 텍스트를 분석하고, 공감하며 위로와 격려를 해주는 **AI 공감 다이어리**를 구현해 보겠습니다.

5W1H 프레임워크로 제품 요구사항 정리하기

실제 개발팀의 PM은 제품 기획 과정에서 5W1H 프레임워크를 통해 팀이 같은 목표와 방향을 공유하도록 제품 요구사항을 구체화합니다. 이는 AI와 함께 애플리케이션을 구현할 때도 마찬가지인데, 2장에서도 PRD를 작성할 때 5W1H 프레임워크를 기반으로 했었죠.

PM은 Product Manager의
앞글자를 딴 약어로, 제품 기획
관리자를 가리킵니다.



여러 에이전트를 운용할 때 역시 각각의 역할^{Who}과 작업 방식^{How}을 구체적으로 정의하는 것이 중요합니다. 5W1H 프레임워크에 근거해 'AI 공감 다이어리'의 요구사항을 다음과 같이 정리해 보겠습니다.

What: AI 공감 다이어리 – 사용자가 오늘 있었던 일을 한 줄로 쓰면 AI가 감정을 분석하고 공감과 위로를 해주는 일기 앱

Why: 사용자의 하루를 기록하고 정서적 지원을 제공하여 마음의 안정을 돕기 위함

When: 지금 바로 개발 시작하여 즉시 사용 가능한 완성품 제작

Where: 웹 브라우저에서 실행, 사용자의 로컬 환경에서 작동

How:

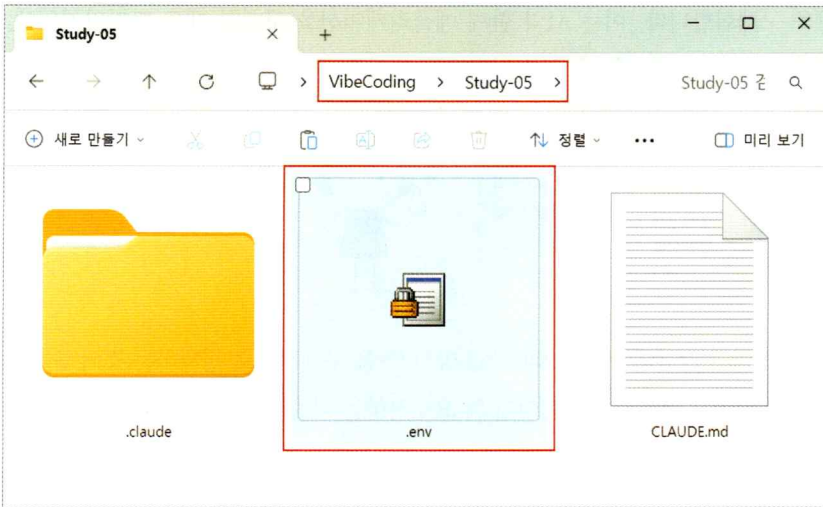
- OpenRouter API와 DeepSeek V3.1 무료 모델을 활용하여 감정 분석 및 공감 메시지 생성
- 따뜻하고 편안한 느낌의 UI 디자인 적용
- API 키는 현재 프로젝트 폴더의 '.env' 파일에서 읽어와 사용
- 최종 결과물은 'index.html' 단일 파일로 제작
- 철저한 테스트를 통해 안정적인 버전 완성

Who:

- 백엔드 개발자: API 연동, 감정 분석과 공감 메시지를 생성하는 기능 구현
- 프론트엔드 개발자: 따뜻하고 편안한 느낌의 일기장 UI 제작
- 품질 보증 엔지니어: 여러 상황을 테스트하고 문제 발견 시 완전히 해결될 때까지 수정

AI 공감 다이어리 애플리케이션을 개발하기 위해서는 백엔드, 프론트엔드, 품질 보증 에이전트가 필요합니다. 오픈라우터의 API를 사용해 AI 모델을 애플리케이션에 연동하고, 서브에이전트를 활용해 구현해 봅시다.

01 데스크톱 파일 탐색기에서 오픈라우터 API 키를 불러오기 위해 'Study-04' 폴더에서 '.env' 파일을 복사한 뒤, 'Study-05' 폴더에 붙여 넣습니다.

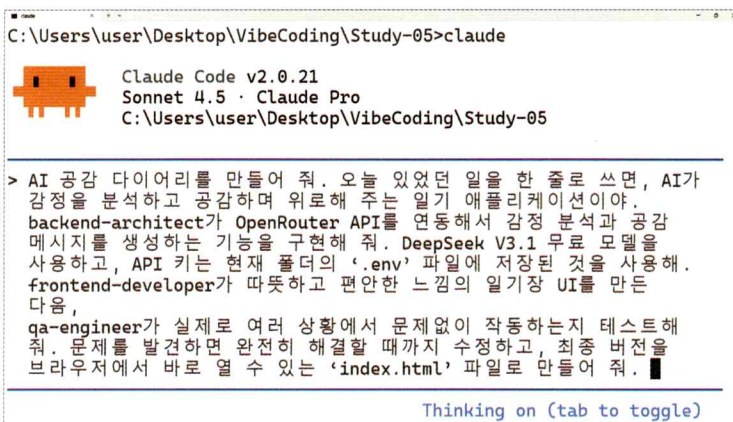


note 'Study-04' 폴더의 '.env' 파일에는 오픈라우터의 API 키가 저장되어 있습니다.

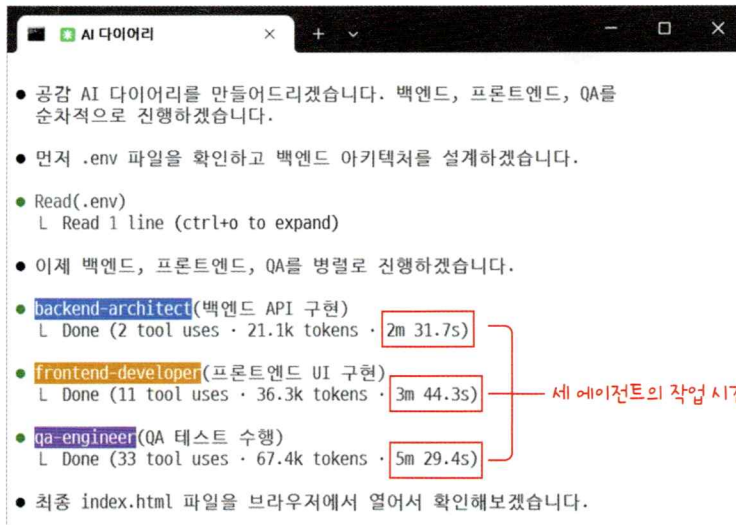
02 클로드 코드에 'AI 공감 다이어리' 애플리케이션의 기능을 설명하고, 필요한 서브에이전트와 각 에이전트의 역할과 구현 방식을 입력합니다.

프롬프트

AI 공감 다이어리를 만들어 줘. 오늘 있었던 일을 한 줄로 쓰면, AI가 감정을 분석하고 공감하며 위로해 주는 일기 애플리케이션이야.
 backend-architect가 OpenRouter API를 연동해서 감정 분석과 공감 메시지를 생성하는 기능을 구현해 줘. DeepSeek V3.1 무료 모델을 사용하고, API 키는 현재 폴더의 '.env' 파일에 저장된 것을 사용해.
 frontend-developer가 따뜻하고 편안한 느낌의 일기장 UI를 만든 다음,
 qa-engineer가 실제로 여러 상황에서 문제없이 작동하는지 테스트해 줘. 문제를 발견하면 완전히 해결할 때까지 수정하고, 최종 버전을 브라우저에서 바로 열 수 있는 'index.html' 파일로 만들어 줘.



03 터미널에서 세 에이전트가 지시한 대로 작업하는 것을 확인할 수 있습니다.



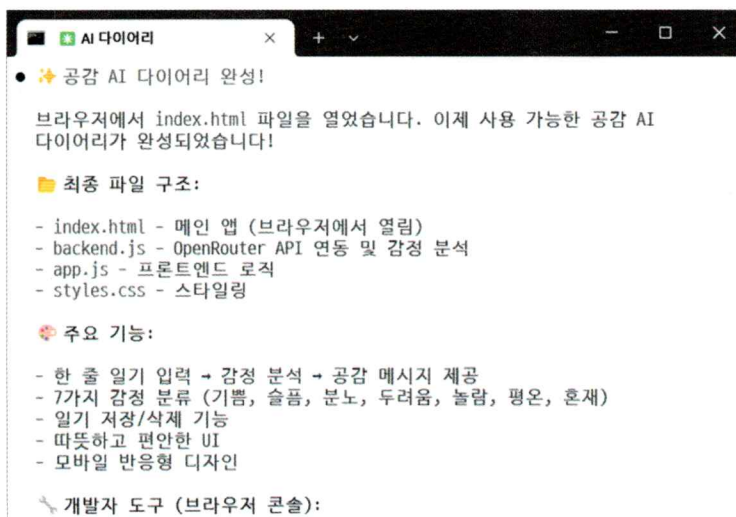
세 에이전트의 작업 시간을 확인해 보세요.

+ 여기서 잠깐 에이전트 작업 시간이 서로 다른 이유

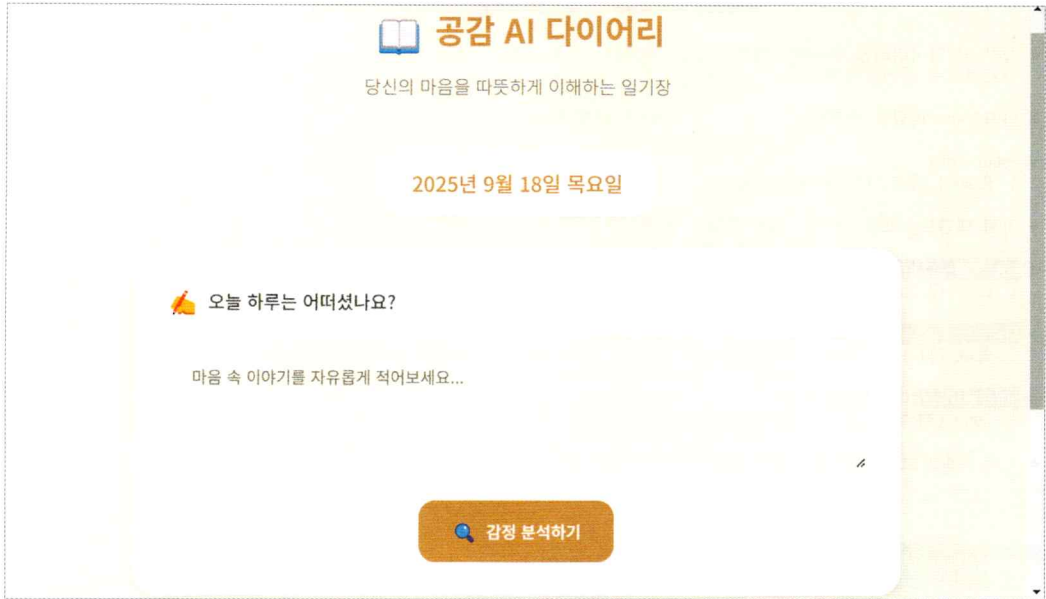
각 에이전트의 작업 시간이 조금씩 다른 것은 자연스러운 현상입니다. 품질 보증 엔지니어 에이전트의 작업 시간이 가장 오래 걸린 이유는 앞서 다른 두 에이전트가 생성한 결과물을 모두 검토하고, 여러 시나리오를 테스트하며, 발견한 오류를 수정하는 과정까지 포함하기 때문입니다.

실제 개발팀에서도 품질 보증^{QA}은 가장 시간이 많이 걸리는 과정입니다. 만약 실행 시간이 너무 길게 느껴진다면 [Esc] 키를 눌러 중단하고, 더 간단한 테스트만 요청해 작업 범위를 줄일 수도 있습니다.

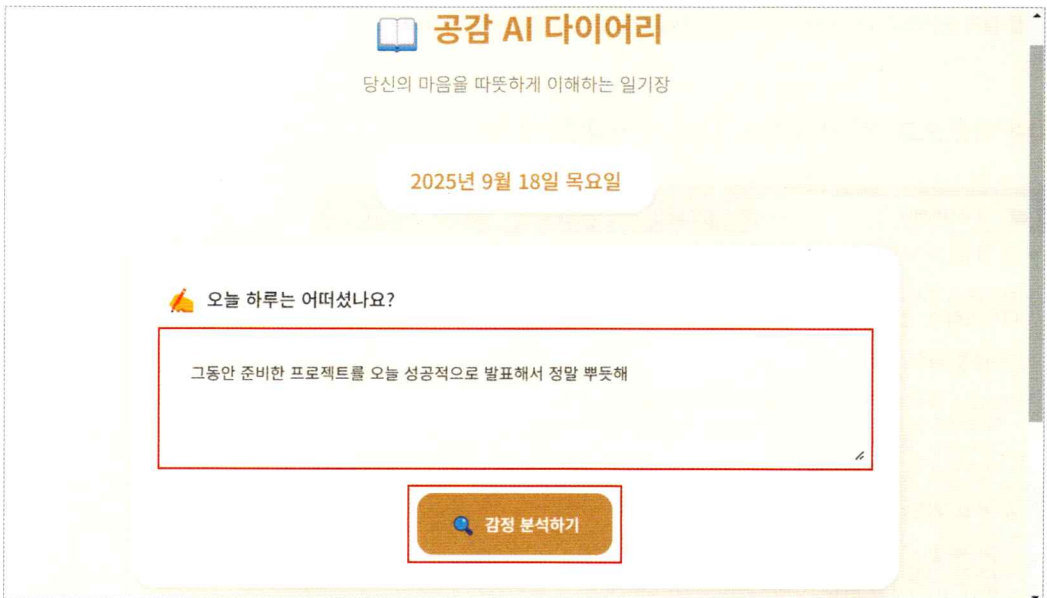
04 최종 보고서를 보고 어떤 기능이 준비되었는지 확인합니다.



05 'Study-05' 폴더에서 'index.html' 파일을 클릭해 실행하겠습니다. AI 공감 다이어리가 완성된 모습입니다.



06 애플리케이션이 텍스트 속 감정을 잘 분석하는지 테스트해 보겠습니다. 다음과 같이 입력하고, [감정 분석하기] 버튼을 클릭하세요.



07 뿌듯한 감정을 잘 탐지하고, 공감의 글을 덧붙여 줍니다.

감정 분석 결과



기쁨
감정 강도: 9/10

❤️ 공감 메시지

오랜 준비 끝에 성공적으로 발표를 마치셨군요. 정말 뿌듯한 마음이 느껴지네요. 그동안의 노력이 결실을 맺은 순간이었을 거예요. 앞으로도 이런 기쁜 날들이 더 많아지길 바랍니다.

일기 저장하기

note 다양한 감정을 담은 일기를 써서 애플리케이션이 감정을 잘 탐지하는지 테스트해 보세요.

08 작성한 감정 일기와 감정 분석 결과는 시간 순서대로 저장됩니다.

**지난 일기들**

 **접기**

2025년 9월 18일 오후 01:45

 슬픔 (5/10)

친구랑 약속이 있었는데 갑자기 취소되어서 혼자 시간을 보냈어

 삭제

2025년 9월 18일 오후 01:40

 기쁨 (9/10)

그동안 준비한 프로젝트를 오늘 성공적으로 발표해서 정말 뿌듯해

 삭제

당신의 모든 감정이 소중한입니다 ✨

note 실행 결과가 책과 다르다면 코드에 원하는 애플리케이션의 요구사항을 구체적으로 입력해 수정할 수 있습니다.

에이전트 팀과 완성하는 애플리케이션, ‘PDF 요약 AI’

이번에는 다섯 개의 서브에이전트를 총동원해 PDF 파일을 입력받아 텍스트를 추출하고, 이를 요약, 정리, 분석하는 애플리케이션을 만들겠습니다. 앞서 만들어 둔 AI 에이전트를 활용해 보겠습니다. **PDF 문서 요약 AI** 애플리케이션 개발을 위해 필요한 각 에이전트들의 역할과 이름, 임무는 다음과 같습니다.

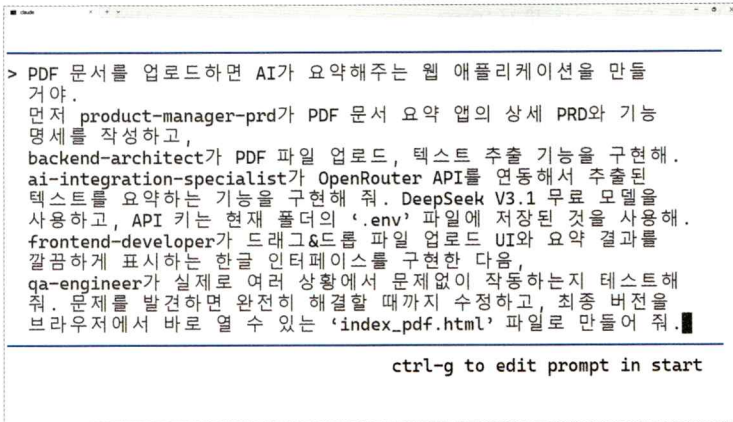
역할	에이전트 명	작업 할당
제품 기획 관리자	product-manager-prd	PDF 문서 요약 앱의 상세 PRD와 기능 명세를 작성
백엔드 개발자	backend-architect	PDF 파일 업로드, 텍스트 추출 기능 구현
AI 통합 전문가	ai-integration-specialist	OpenRouter API를 연동해서 추출된 텍스트를 요약하는 기능 구현. 무료 모델을 사용하고, API 키는 .env 파일에 저장된 것을 사용
프론트엔드 개발자	frontend-developer	드래그&드롭 파일 업로드 UI와 요약 결과를 깔끔하게 표시하는 인터페이스를 구현
품질 보증 엔지니어	qa-engineer	다양한 PDF 형식으로 전체 시스템을 철저히 테스트하고 버그를 수정

note 클로드 코드와 대화할 때, 프롬프트에는 항상 서브에이전트의 정확한 실제 이름을 사용해야 하며, '/agents' 명령어로 생성한 서브에이전트의 목록을 확인할 수 있습니다.

01 클로드 코드를 실행해 ‘PDF 요약 AI’ 애플리케이션의 기능을 설명하고, 각 서브에이전트의 역할과 구현 방식을 입력합니다.

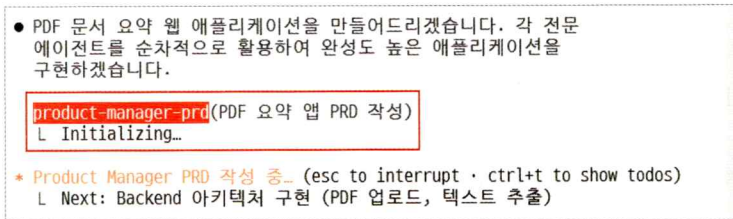
프롬프트

PDF 문서를 업로드하면 AI가 요약해주는 웹 애플리케이션을 만들 거야.
먼저 product-manager-prd가 PDF 문서 요약 앱의 상세 PRD와 기능 명세를 작성하고,
backend-architect가 PDF 파일 업로드, 텍스트 추출 기능을 구현해.
ai-integration-specialist가 OpenRouter API를 연동해서 추출된 텍스트를 요약하는 기능을 구현해 줘.
DeepSeek V3.1 무료 모델을 사용하고, API 키는 현재 폴더의 '.env' 파일에 저장된 것을 사용해.
frontend-developer가 드래그&드롭 파일 업로드 UI와 요약 결과를 깔끔하게 표시하는 한글 인터페이스를 구현한 다음,
qa-engineer가 실제로 여러 상황에서 문제없이 작동하는지 테스트해 줘. 문제를 발견하면 완전히 해결할 때까지 수정하고, 최종 버전을 브라우저에서 바로 열 수 있는 'index_pdf.html' 파일로 만들어 줘.

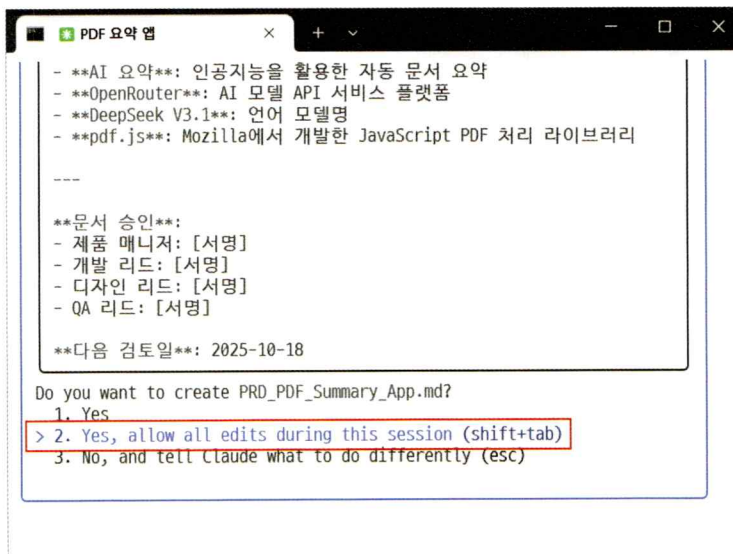


note 'AI 공감 다이어리' 애플리케이션의 실행 파일 이름과 중복되지 않도록 최종 실행 파일 이름을 'index_pdf.html'로 지정했습니다.

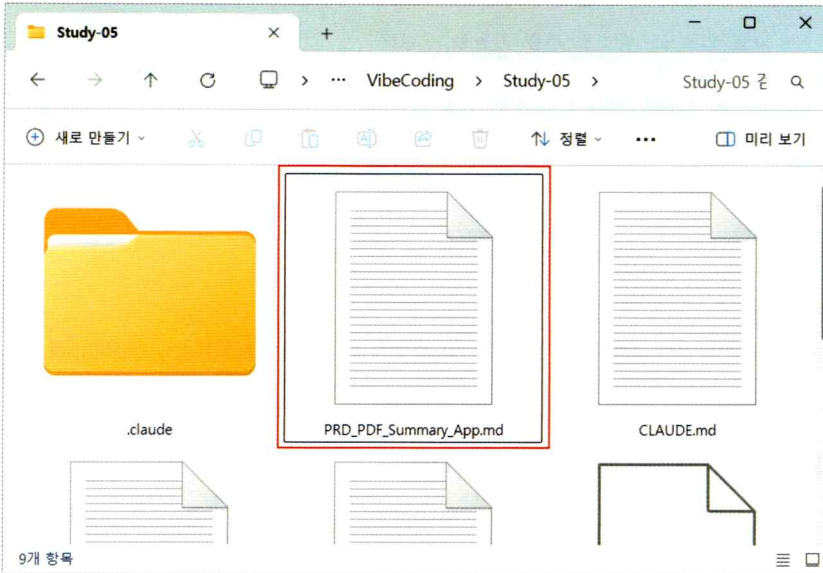
02 제품 기획 관리자가 PRD를 만드는 과정부터 시작합니다.



03 PRD에 이상이 없으면 '2. Yes, allow all edits during this session'을 선택해 진행합니다.



04 'Study-05' 폴더에 프로젝트를 위한 PRD 문서 'PRD_PDF_Summary_App.md'가 생성된 것을 확인합니다.

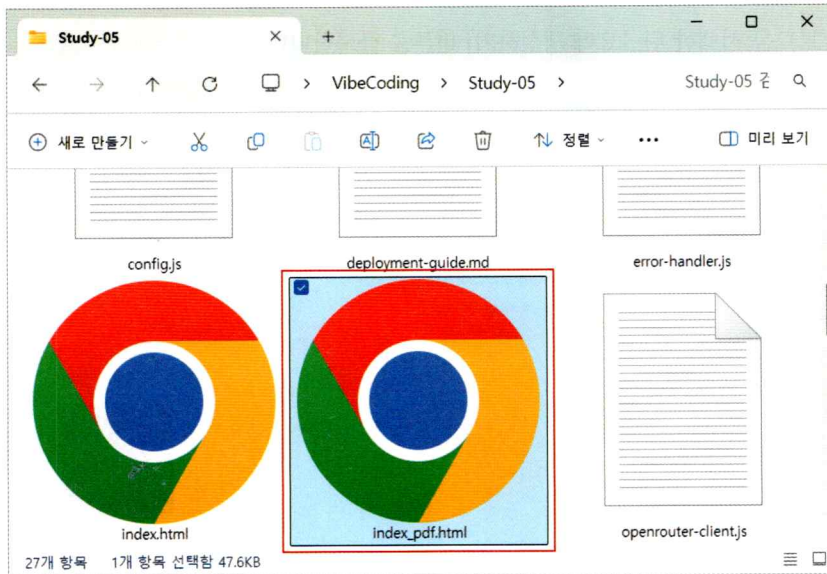


note 파일명은 책과 다를 수 있습니다.

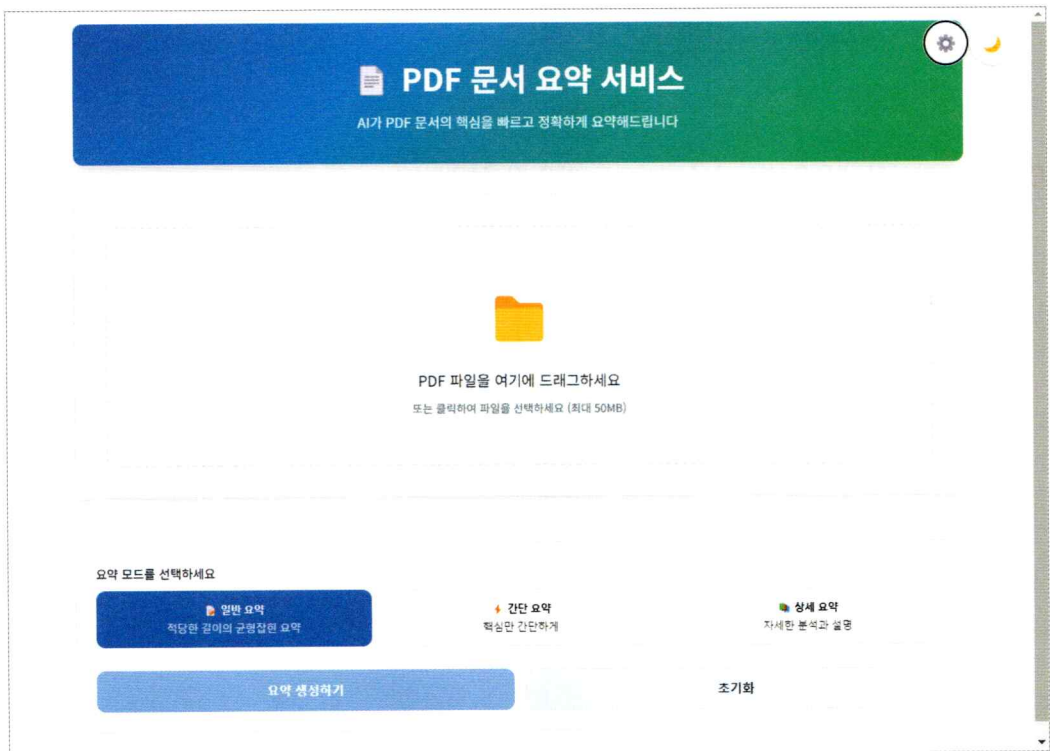
05 다섯 개의 에이전트가 순차적으로 작업을 수행하고, 웹 애플리케이션이 완성됩니다.



06 'Study-05' 폴더에 새로 생성된 'index_pdf.html' 파일을 찾아 실행합니다.



07 이와 같이 PDF 문서 요약 서비스앱이 실행되면 완성입니다. 원하는 결과가 나오지 않았다면, 현재 상황을 클로드 코드에게 다시 입력하여 수정, 보완해 나가면 됩니다.



08 테스트를 위해 깃허브 자료실(<https://github.com/taehojo/vibecoding>)의 ‘입력예제’ 폴더에서 ‘한글맞춤법.pdf’ 파일을 다운로드하세요. 애플리케이션으로 돌아와 ‘한글맞춤법.pdf’ 파일을 업로드하고 [일반 요약] 모드를 선택한 뒤, [요약 생성하기] 버튼을 클릭합니다.



09 PDF 파일의 내용을 요약하여 제시하는 것을 확인합니다.



10 이번에는 [간단 요약] 모드를 선택하고 [요약 생성하기] 버튼을 클릭해 봅니다. 조금 다른 요약 결과를 보여줍니다.

요약 결과

한글 맞춤법은 일상생활에서 한글을 올바르게 표기하기 위한 규범으로, 국립국어원에서 제정한 대원칙은 “표준어를 소리대로 적되, 어법에 맞도록 함”입니다. 이는 발음을 기본으로 하면서도 의미 파악을 쉽게 하기 위해 어법을 고려하는 것입니다. 주요 내용은 다음과 같습니다:

1. ****핀소리와 거센소리 표기****: 핀소리는 독립된 음소이므로 정확하게 표기해야 합니다. 예를 들어, '가끔'을 '가금'으로 쓰는 것은 잘못입니다. 또한, '어떻게'와 '어떡해'는 각각 방법이나 상태를 묻는 부사와 '어떻게 해'가 줄어든 말로 구별해야 합니다.
2. ****사이시옷의 이해****: 사이시옷은 합성어를 만들 때 두 단어 사이에 들어가는 스으로, 순우리말과 순우리말 또는 순우리말과 한자어가 결합할 때 사용됩니다. 예를 들어, '코'와 '동'이 만나면 '콧동'이 됩니다. 단, 한자어와 한자어가 결합할 때는 원칙적으로 사이시옷을 쓰지 않습니다.
3. ****띄어쓰기의 원칙****: 한글 띄어쓰기의 기본 원칙은 단어별로 띄어 쓰는 것입니다. 조사는 앞말에 붙여 쓰고, 의존명사와 보조용언은 띄어 씁니다. 예를 들어, '할 수 있다'에서 '수'는 의존명사이므로 앞말과 띄어 씁니다.
4. ****자주 혼동하는 표기들****: '왓치'와 '왓치', '되'와 '돼', '금세'와 '금새' 등은 자주 혼동되는 표현입니다. '왓치'는 '왜인지'가 줄어든 말이고, '되'와 '돼'는 '하'를 넣어 자연스러우면 '돼'를 씁니다.
5. ****준말과 본말의 관계****: 준말을 쓸 때는 본래의 형태를 생각해야 합니다. 예를 들어, '뭐라고'는 '무엇이라고'의 준말이고, '거예요'는 '것이예요'의 준말입니다.

note 다양한 요약 모드로 작업해 보세요.

지금까지 우리는 다섯 개의 AI 서브에이전트를 구성해 실제 개발팀처럼 협업하는 과정을 살펴보았습니다. 각 에이전트가 맡은 역할에 따라 기획부터 개발, 디자인, 검증까지 자동으로 진행되며, 혼자서도 팀 단위 프로젝트를 완성할 수 있음을 확인했습니다. 이제 여러분은 단순히 코드를 작성하는 개발자가 아니라, 여러 AI 에이전트를 조율해 하나의 제품을 완성하는 AI 프로젝트 매니저가 된 것입니다.

마무리

▶ 5가지 키워드로 정리하는 핵심 포인트

- **제품 기획 관리자**는 제품의 목표, 기능, 사용자 요구사항을 정의하고 전체 개발 일정을 관리합니다.
- **백엔드 개발자**는 서버, 데이터베이스 설계와 API 연동 등 애플리케이션의 핵심 로직을 구현하는 기술 담당자입니다.
- **프론트엔드 개발자**는 사용자가 보는 화면과 인터페이스를 설계하고 구현하는 UI 개발자입니다.
- **품질 보증 엔지니어**는 기능과 성능을 테스트해 버그를 수정하고 서비스의 안정성을 확보하는 검증 전문가입니다.
- AI를 도입한 개발 과정에서 AI 모델을 연동, 최적화하기 위해 기술 통합 담당자인 **AI 통합 전문가**가 필요합니다.

▶ 확인 문제

1. AI 공감 다이어리 개발에서 백엔드 개발과 프론트엔드 개발보다 품질 보증(QA) 테스트 시간이 더 오래 걸린 이유로 가장 적절한 것을 고르세요.
 - ① QA 에이전트가 더 느린 AI 모델을 사용하기 때문이다.
 - ② QA 에이전트는 이전 에이전트들의 작업 결과를 모두 검토해야 하기 때문이다.
 - ③ QA 에이전트만 인터넷 연결이 필요하기 때문이다.
 - ④ QA 에이전트는 파일 읽기 권한이 없기 때문이다.

2. PDF 요약 도구에서 제품 기획 관리자 에이전트가 생성한 PRD를 다른 에이전트들이 참조하는 방식으로 옳지 않은 것을 고르세요.

- ① PRD가 프로젝트 폴더에 생성되어 각 에이전트들이 접근한다.
- ② 에이전트들이 순차적, 혹은 병렬로 작업하며 PRD를 참조하게 된다.
- ③ PRD 파일 경로를 에이전트별로 지정하지 않아도 된다.
- ④ PRD는 '.claude/agents/' 폴더에 자동으로 복사되어 공유된다.

